UNIVERSIDADE FEDERAL DO PIAUÍ
CENTRO DE CIÊNCIAS DA NATUREZA
DEPARTAMENTO DE COMPUTAÇÃO

EQUIPE FITOSFERA

Lucas Vilarinho
Gabriel Araújo
Matheus Carneiro

FitoSfera

nó sensor de temperatura, umidade e luz que entrega suas leituras por Bluetooth Low Energy a quem cuida de um pequeno jardim doméstico

---

Relatório técnico da AP2

Ciclo de Projeto 2 — integração à infraestrutura de comunicação e formação do conjunto de dados

Tópicos em Sistemas Computacionais I — Internet das Coisas e AIoT

2026.2 • Prof. Atos Apollo Silva Borges

Teresina

25 de outubro de 2026

FitoSfera

Relatório técnico do Ciclo de Projeto 2 — integração à infraestrutura de comunicação e formação do conjunto de dados, apresentado como instrumento da AP2 da disciplina Tópicos em Sistemas Computacionais I — Internet das Coisas e AIoT, 2026.2, do Departamento de Computação da Universidade Federal do Piauí.

Universidade Federal do Piauí
Centro de Ciências da Natureza
Departamento de Computação

Docente: Prof. Atos Apollo Silva Borges

Teresina, Piauí
25 de outubro de 2026

Resumo

Quem cultiva plantas em casa decide quando regar ou mudar um vaso de lugar pela aparência da planta, e só percebe o problema quando ela já murchou. O FitoSfera propõe trocar essa avaliação a olho por medições do ambiente, comparadas com o que cada espécie exige. Este relatório documenta o segundo ciclo do projeto: a ligação do nó sensor a quem guarda o dado e a formação do conjunto de dados. O nó usa um ESP32 com um DHT11, para temperatura e umidade do ar, e um sensor de luz BH1750, que mede até cerca de 120 mil lux. A cada cinco minutos, ele envia um pacote de 19 bytes por Bluetooth Low Energy, com número de sequência, um identificador que muda a cada reinício e a luz acumulada desde que ligou, de modo que uma desconexão não apaga a luz recebida. A equipe escolheu Bluetooth Low Energy e não WiFi porque o central do produto é o celular do usuário, que já está na casa. Um gravador escrito pela equipe recebeu os pacotes durante 24 horas de campanha, gravou cada um antes de confirmá-lo e produziu um conjunto de dados com 97,6 % de completude. Dezessete requisitos foram verificados por dezoito casos de teste, e 15 foram atendidos, 1 em parte e 1 ficou sem verificação. O ponto fraco é que o nó não guarda temperatura e umidade quando o central está longe. No próximo ciclo entram uma fila local, o pareamento e o aplicativo como central.

Palavras-chave: internet das coisas; ESP32; Bluetooth Low Energy; monitoramento de plantas; conjunto de dados.

Abstract

People who grow plants at home decide when to water a plant or move a pot based on how it looks, and notice problems only after it has wilted. FitoSfera proposes replacing this judgement by eye with measurements of the environment, compared with what each species needs. This report documents the project's second cycle: connecting the sensor node to whoever stores the data, and building the dataset. The node uses an ESP32 with a DHT11 for air temperature and humidity and a BH1750 light sensor that measures up to about 120 thousand lux. Every five minutes it sends a 19-byte packet over Bluetooth Low Energy, with a sequence number, an identifier that changes at every reboot and the light accumulated since power-up, so that a disconnection does not erase the light received. The team chose Bluetooth Low Energy over WiFi because the product's central is the user's phone, which is already in the house. A recorder written by the team received the packets during 24 hours of data collection, stored each one before acknowledging it and produced a dataset with 97.6 % completeness. Seventeen requirements were verified by eighteen test cases, and 15 were met, 1 partially and 1 remained unverified. The weak point is that the node does not keep temperature and humidity while the central is away. The next cycle adds a local queue, pairing and the app as the central.

Keywords: internet of things; ESP32; Bluetooth Low Energy; plant monitoring; dataset.

Lista de ilustrações

Figura 1 – Arquitetura do FitoSfera na AP2: o que rodou na campanha (contínuo) e o que fica para depois ou só serve à depuração (tracejado) 16
Figura 2 – Esquema elétrico do protótipo da AP2. As cores nos rótulos são as dos fios 22
Figura 3 – Caminho do dado durante a campanha de coleta. A seta de volta é a confirmação, escrita depois de gravar 30
Figura 4 – Temperatura, umidade relativa e luz durante a campanha, com as lacunas à vista 43

Lista de tabelas

Tabela 1 – Do produto ao nó sensor . . . 10
Tabela 2 – Requisitos funcionais do Ciclo de Projeto 2 — integração à infraestrutura de comunicação e formação do conjunto de dados . . . 13
Tabela 3 – Requisitos não funcionais . . . 14
Tabela 4 – Requisitos da AP1 que mudaram ou saíram . . . 14
Tabela 5 – Rastreabilidade entre requisitos, implementação e testes . . . 15
Tabela 6 – Lista de materiais do protótipo . . . 21
Tabela 7 – Tabela de ligações . . . 22
Tabela 8 – Matriz de decisão de conectividade . . . 28
Tabela 9 – Marcas que mudaram desde a AP1 . . . 29
Tabela 10 – Onde roda cada peça, e o que acontece se ela parar . . . 30
Tabela 11 – Contrato de dados da telemetria (versão 2) . . . 31
Tabela 12 – Contrato GATT do nó (o equivalente do contrato de tópicos MQTT) . . . 32
Tabela 13 – Linha do rádio: estimada na AP1 *versus* medida na AP2 . . . 33
Tabela 14 – Ficha da campanha de coleta . . . 34
Tabela 15 – Qualidade do conjunto de dados coletado . . . 35
Tabela 16 – Lacunas do conjunto de dados . . . 35
Tabela 17 – Descrição do conjunto de dados (`dados/mensagens_limpo.csv`) . . . 36
Tabela 18 – Casos de teste e resultados . . . 40
Tabela 19 – Mapa de barramento do protótipo . . . 50
Tabela 20 – Orçamento de energia do nó (alimentação USB) . . . 51
Tabela 21 – CT-01 — período de leitura . . . 53
Tabela 22 – CT-02 — luz sob sol direto . . . 53
Tabela 23 – CT-05 — formato CSV na serial . . . 54
Tabela 24 – CT-06 — falha de sensor . . . 54
Tabela 25 – CT-07 — corrente média na USB . . . 55
Tabela 26 – CT-10 — pinos com restrição . . . 55
Tabela 27 – CT-12 — pacote no contrato . . . 56
Tabela 28 – CT-13 — medir agora e assinatura . . . 56
Tabela 29 – CT-14 — confirmação . . . 57
Tabela 30 – CT-15 — queda do enlace . . . 57
Tabela 31 – CT-16 — o gravador para . . . 58
Tabela 32 – CT-17 — o nó reinicia . . . 58
Tabela 33 – CT-18 — o nó morre sem avisar . . . 59
Tabela 34 – CT-19 — antes do NTP: a hora das leituras . . . 59
Tabela 35 – CT-20 — luz através de uma desconexão . . . 60
Tabela 36 – CT-21 — alcance . . . 60
Tabela 37 – CT-22 — longa duração: a campanha . . . 61
Tabela 38 – CT-23 — tamanho do pacote . . . 61
Tabela 39 – Uso de IA generativa neste ciclo . . . 62
Tabela 40 – Checklist de prontidão e o que a campanha mostrou . . . 64
Tabela 41 – Diário da campanha de coleta . . . 66
Tabela 42 – Eventos observados durante a coleta . . . 67

Lista de abreviaturas e siglas

ADC	*Analog-to-Digital Converter* — conversor analógico-digital
AIoT	*Artificial Intelligence of Things*
ATT	*Attribute Protocol* — protocolo de atributos do BLE
BLE	*Bluetooth Low Energy*
CSV	*Comma-Separated Values* — valores separados por vírgula
DLI	*Daily Light Integral* — luz integrada do dia
GATT	*Generic Attribute Profile* — perfil genérico de atributos do BLE
GPIO	*General Purpose Input/Output*
I^2C	*Inter-Integrated Circuit*
IoT	*Internet of Things* — internet das coisas
LED	*Light Emitting Diode* — diodo emissor de luz
MQTT	*Message Queuing Telemetry Transport*
MTU	*Maximum Transmission Unit* — maior unidade de transmissão
NTP	*Network Time Protocol*
RF	Requisito funcional
RNF	Requisito não funcional
RSSI	*Received Signal Strength Indicator* — intensidade do sinal recebido
UART	*Universal Asynchronous Receiver/Transmitter*
UR	Umidade relativa do ar
USB	*Universal Serial Bus*
UUID	*Universally Unique Identifier*

Sumário

1 O problema . . . 9
1.1 Contexto . . . 9
1.2 Pergunta de projeto . . . 10
1.3 Quem usa o sistema, e o que muda para essa pessoa . . . 10
1.4 Escopo deste ciclo . . . 10
1.5 Soluções existentes . . . 11
2 Requisitos e critérios de aceitação . . . 12
2.1 Requisitos funcionais . . . 13
2.2 Requisitos não funcionais . . . 14
2.3 Restrições . . . 15
2.4 Rastreabilidade . . . 15
3 Arquitetura do sistema . . . 16
3.1 Visão em camadas . . . 16
3.2 Componentes e decisões de projeto . . . 16
3.3 Mapa de barramento . . . 20
3.4 Orçamento de energia . . . 20
3.5 Lista de materiais . . . 21
4 Implementação . . . 22
4.1 Montagem . . . 22
4.2 Estrutura do firmware . . . 23
4.3 Firmware de comunicação . . . 24
4.4 O gravador . . . 25
4.5 Tratamento do sinal . . . 26
4.6 Comportamento em falha . . . 26
4.7 Repositório . . . 26
5 Conectividade e conjunto de dados . . . 28
5.1 Tecnologia de comunicação . . . 28
5.2 Caminho do dado, do sensor ao arquivo . . . 29
5.3 Contrato de dados . . . 30
5.4 Contrato GATT e garantias de entrega . . . 32
5.5 Custo real da comunicação . . . 33
5.6 A campanha de coleta . . . 33
5.7 Qualidade do conjunto de dados . . . 34
5.8 Descrição do conjunto de dados . . . 35
6 Verificação e testes . . . 38
6.1 Casos de teste . . . 40
6.2 Testes de robustez: os testes mínimos da AP2 . . . 41
6.3 O que falhou . . . 42
7 Resultados . . . 43
7.1 O sistema em operação . . . 43
7.2 Requisitos atendidos . . . 44
7.3 Discussão . . . 45
8 Limitações e próximo ciclo . . . 46
8.1 Limitações conhecidas . . . 46
8.2 O que entra no próximo ciclo . . . 46

8.3 Divisão do trabalho 47

Referências 48

Apêndices 49
APÊNDICE A Mapa de barramento 50
Decisões de barramento 50
APÊNDICE B Orçamento de energia 51
Quatro linhas de decisão 52
Se a conta atravessar tensões diferentes 52
Se o projeto for alimentado por painel 52
Quem está dimensionando este sistema 52
APÊNDICE C Plano de testes 53
APÊNDICE D Declaração de uso de inteligência artificial generativa 62
APÊNDICE E Checklist de prontidão para a campanha 64
O que o checklist não pegou 65
APÊNDICE F Registro da campanha de coleta 66
Diário da campanha 66
Registro de eventos para rotulagem 66
Saída do relatório de qualidade 67

1 O problema

Rubrica da AP2: justificativa técnica das decisões — 25 %

1.1 Contexto

Pequenos jardins, hortas em varanda e vasos dentro de casa são cuidados por pessoas que, em geral, não têm formação agronômica. Elas decidem quando regar, se devem mover um vaso para perto da janela ou se a planta está sofrendo com o calor olhando para a planta. Isso tem dois problemas. Primeiro, quando a planta mostra o sintoma (folha murcha, amarelada ou queimada), o dano já aconteceu, porque ela responde ao ambiente com horas ou dias de atraso. Segundo, o que é adequado depende da espécie: a mesma umidade que faz bem a uma samambaia pode fazer mal a uma suculenta no vaso ao lado.

Em Teresina a situação é mais difícil. A temperatura fica acima de 30 °C em boa parte do ano e, entre agosto e outubro, na estação seca, a umidade relativa do ar cai bastante. Plantas trazidas de outras regiões sofrem justamente com essas duas grandezas, e a avaliação a olho não percebe a mudança a tempo.

A ideia do projeto surgiu de uma conversa de um dos integrantes com a avó, Teresinha, que cuida de plantas no jardim de casa. Ela relatou duas dificuldades: lembrar quando foi a última rega de cada planta e saber quais são as condições adequadas para cada uma. O integrante conhece de perto a rotina dela e mora perto, o que torna o jardim um lugar acessível para instalar o nó.

Onde o nó fica, e onde está o primeiro ponto com internet

Na AP2 o nó saiu da bancada. Ele foi instalado em dois lugares, sempre a cerca de 1 m do chão e ligado a um carregador USB de 5 V na tomada: num quarto fechado, sobre uma mesa, e num jardim aberto, fora da casa. A campanha de coleta rodou num quarto, junto a uma janela aberta, com sol direto em parte do dia, com o nó a 50 cm do chão. O primeiro equipamento capaz de receber as leituras e levá-las adiante é o *central* BLE: na campanha, o notebook que rodou o gravador, a cerca de 5 m do nó (Seção 5.2); no produto, o celular de quem cuida do jardim.

A distância máxima com comunicação de sucesso foi de cerca de 20 a 25 m dentro da casa, com várias paredes no caminho, e de cerca de 45 a 50 m no ambiente aberto, com uma parede só (**CT-21**). A casa tem WiFi, e o roteador seria o primeiro ponto com internet numa solução por WiFi, mas o projeto não depende dele: no BLE o roteador nem entra no caminho do dado.

Daí sai o requisito de alcance (**RNF-06**). Num jardim doméstico, o celular de quem cuida das plantas fica a no máximo cerca de 10 m do nó, quase sempre com uma ou duas paredes no caminho. O enlace medido alcança o dobro disso dentro de casa, então o produto não precisa de infraestrutura nova.

1.2 Pergunta de projeto

> **Como podemos** informar agricultores domésticos e donos de jardim, com base em medições de temperatura, umidade e luz, **para que** cuidem de cada planta segundo o que a espécie exige, **de modo que** saibam quando regar ou reposicionar um vaso antes de a planta mostrar sinais de dano**?**

1.3 Quem usa o sistema, e o que muda para essa pessoa

O usuário é um agricultor doméstico ou dono de jardim, como a Teresinha. Pode ser alguém que só quer manter as plantas vivas sem muito esforço, ou alguém que cultiva hortaliças e frutos e quer acompanhar o tempo até a colheita. Hoje essa pessoa decide pela aparência da planta e pela memória ("acho que reguei anteontem"). Com o FitoSfera, ela passa a contar com medições do ambiente onde a planta está, comparadas pelo aplicativo com o que a espécie exige, avisos acionáveis ("mover a planta Y para um lugar mais iluminado") e um registro de cada planta.

Quatro características do produto determinam o que o nó precisa fazer. É delas que saem os requisitos do Capítulo 2:

Tabela 1 – Do produto ao nó sensor

Característica do produto	**Consequência para o nó**
Monitora umidade, temperatura e luz	Três grandezas. Na AP2 a luz deixou de ser claro/escuro e passou a ser medida em lux (BH1750), porque o que a espécie exige é uma quantidade de luz por dia, e não um estado (**DP-2**).
Compara com o que cada espécie exige	A comparação mudou para o aplicativo, que conhece a planta cadastrada; o nó só mede e entrega (**DP-3**).
Fala com o celular de quem cuida do jardim	O celular é o central: o nó não precisa de roteador, senha de rede nem servidor. O enlace é BLE (**DP-11**).
Registra o que aconteceu, com hora	O nó não tem relógio: quem carimba a hora é o central, na chegada do pacote (**DP-10**). A luz de um período longo sem comunicação não se perde, porque o nó a acumula (**DP-8**).

Fonte: Elaborada pela equipe.

As plantas respondem ao ambiente em horas. Na AP1 o nó lia os sensores a cada 2 s, mais de cem vezes o necessário. Na AP2 o período passou a 300 s (**DP-9**): continua dez vezes mais rápido que o fenômeno e reduz a 288 por dia os pacotes que o rádio precisa entregar.

1.4 Escopo deste ciclo

Está neste ciclo (Módulo 2):

- nó com ESP32 DevKit V1, DHT11 (temperatura e umidade do ar) e BH1750 em I^2C (luz em lux, até cerca de 120.000 lux);

- enlace BLE com um serviço próprio: uma característica de leitura (notificação de um pacote binário de 19 B) e uma de comando (medir agora e confirmar recebimento);
- confirmação de recebimento pela aplicação, luz acumulada no nó e identificação de reinícios e de pacotes perdidos;
- um gravador BLE em Python, que faz o papel do aplicativo durante a campanha e forma o conjunto de dados em CSV;
- a campanha de coleta e a análise de qualidade do conjunto de dados (`qualidade.py`).

Não está neste ciclo, e por quê:

- **Aplicativo FitoSfera falando com o nó:** o aplicativo ainda não tem o adaptador BLE. Na campanha, o gravador cumpre esse papel e segue o mesmo contrato.
- **WiFi, MQTT e *broker*:** a equipe escolheu BLE (**DP-11**). Não há *broker*, testamento nem NTP no nó; os equivalentes no BLE estão na Seção 5.4.
- **Pareamento e segurança:** o nó aceita qualquer central ao alcance. Fica para a AP3 (Capítulo 8).
- **LEDs de aviso no nó:** saíram junto com o módulo fotorresistor. O aviso passa a ser do aplicativo (**DP-3**).
- **Bateria e *deep sleep*:** o nó continua na tomada (Apêndice B).
- **Umidade do solo:** continua sendo a grandeza que melhor indica a rega, mas exige uma sonda por vaso.

1.5 Soluções existentes

- **Sensores comerciais de planta via BLE** (por exemplo, o Xiaomi Flower Care): ficam espetados no vaso e medem umidade do solo, temperatura e luz. Usam a mesma tecnologia que o FitoSfera adotou na AP2, o que confirma que BLE basta para a distância de uma casa. Mas o código é fechado, cada vaso precisa de uma unidade e eles não medem a umidade do ar.
- **Aplicativos de lembrete de cuidado** (por exemplo, Planta): sugerem rega por calendário e por espécie, mas não medem nada. O lembrete é o mesmo na semana mais úmida e na mais seca do ano, que é o erro que o FitoSfera quer evitar.
- **Estações de monitoramento e irrigação agrícola:** medem com precisão, mas exigem *gateway*, assinatura e instalação que não cabem numa varanda.

Nenhuma dessas soluções junta, a baixo custo, medições do ambiente e comparação para cada espécie no celular do usuário. A arquitetura em camadas usada no Capítulo 3 segue Al-Fuqaha et al. (2015).

2 Requisitos e critérios de aceitação

Rubrica da AP2: robustez, testes e documentação — 25 %

Os requisitos abaixo descrevem o que o sistema da AP2 faz, conforme `firmware/src/main.cpp` e `gravador/` no *commit* do Capítulo 4. O nó mudou bastante desde a AP1: o módulo fotorresistor e os três LEDs saíram, o BH1750 entrou, o período passou de 2 s para 300 s e o nó ganhou um enlace BLE. Por isso parte dos requisitos da AP1 mudou e parte saiu (Tabela 4). Os números foram mantidos, para que a rastreabilidade com o relatório anterior não se perca. Cada requisito tem um caso de teste no Capítulo 6, e os procedimentos completos estão no Apêndice C.

2.1 Requisitos funcionais

Tabela 2 – Requisitos funcionais do Ciclo de Projeto 2 — integração à infraestrutura de comunicação e formação do conjunto de dados

ID	O sistema deve...	Critério de aceitação
Sensoriamento (mudaram na AP2)		
RF-01	ler temperatura, umidade do ar e luz a cada 300 s	na campanha, mediana dos intervalos entre leituras do ciclo em 300 s ± 10 %, calculada pelo `qualidade.py`
RF-02	medir a luz em lux, sem saturar sob sol direto	sob sol direto, leitura válida acima de 54.612 lux (o teto do BH1750 na sensibilidade de fábrica)
RF-05	enviar cada leitura também pela porta serial, numa linha CSV com 7 campos	10 de 10 linhas de dados lidas por um leitor de CSV, com 7 campos
RF-06	tentar de novo, a cada 5 s e até 3 vezes, quando um sensor falha; na última, enviar o que houver, com os campos ausentes marcados	com o fio de dados do DHT11 solto: 3 linhas `# nova tentativa` e um pacote com temperatura `0x8000` e umidade `0xFFFF`; o nó não trava e volta a ler quando o fio volta
Comunicação (novos na AP2)		
RF-07	notificar ao central inscrito, a cada leitura, um pacote de 19 B no contrato da Seção 5.3	na campanha, nenhuma linha fora do contrato no `qualidade.py`; os três pacotes de exemplo do README decodificados sem erro
RF-08	medir e enviar na hora quando o central assina a notificação ou escreve `0x01` no comando	10 de 10 pedidos respondidos com um pacote de sequência nova em até 2 s (sem falha de sensor)
RF-09	zerar o contador de segundos sem sucesso só quando chega a confirmação `[0x02, seq]` do pacote em espera	com confirmação, o pacote seguinte traz o contador perto de 300 s; sem confirmação, o contador continua crescendo
RF-10	gravar cada pacote no disco antes de confirmá-lo, e reconectar sozinho depois de uma queda	depois de o nó voltar a anunciar, o gravador reconecta em até 90 s, sem intervenção; todo pacote confirmado está no CSV
RF-11	deixar cada reinício do nó identificável no conjunto de dados	depois de apertar EN: id do reinício novo, sequência de volta a 1 e acumulado de luz de volta a 0; o `qualidade.py` conta 1 reinício, 0 repetidas e 0 perdas falsas
RF-12	acumular a luz no nó, de modo que um período sem comunicação não perca a luz recebida	depois de uma desconexão de pelo menos 30 min, a diferença entre dois acumulados cobre o período inteiro, sem contar em dobro

Fonte: Elaborada pela equipe.

2.2 Requisitos não funcionais

Tabela 3 – Requisitos não funcionais

ID	Atributo	Valor alvo	Como se verifica
RNF-01	Corrente média na entrada USB, com o BLE ligado	$\leq 100\,\text{mA}$ (20 % da USB 2.0)	medidor USB em série, **CT-07**; Apêndice B
RNF-04	Uso de pinos com restrição (inicialização, *flash*, UART0)	nenhum	inspeção contra a folha de dados, **CT-10**
RNF-05	Operação contínua sem travar, com poucas leituras inválidas	$\geq 24\,\text{h}$; campos ausentes $\leq 5\,\%$	campanha, **CT-22**
RNF-06	Alcance do enlace dentro de casa, com paredes no caminho	$\geq 10\,\text{m}$	afastar o nó do central até a comunicação falhar, **CT-21**
RNF-07	Completude do conjunto de dados	$\geq 90\,\%$ das janelas de 300 s	`qualidade.py` sobre o arquivo final, **CT-22**
RNF-08	Tempo até o gravador perceber que o nó caiu	$\leq 10\,\text{s}$ sem energia; $\leq 630\,\text{s}$ com o nó travado e o enlace de pé	`eventos.csv`, **CT-18**
RNF-09	Tamanho do pacote	$\leq 20\,\text{B}$, o que cabe numa notificação com o MTU padrão de 23 B	inspeção do código e do nRF Connect, **CT-23**

Fonte: Elaborada pela equipe.

Tabela 4 – Requisitos da AP1 que mudaram ou saíram

ID	Na AP1	Na AP2, e por quê
RF-01	ler T e UR a cada 2 s	passa a 300 s e inclui a luz (**DP-9**)
RF-02	claro/escuro e LED verde	luz em lux pelo BH1750; o LED saiu (**DP-2**)
RF-03	LED vermelho se $T < 25\,^{\circ}\text{C}$	**retirado**: os LEDs saíram e o aviso passa ao aplicativo (**DP-3**)
RF-04	LED amarelo se UR $< 50\,\%$	**retirado**, pelo mesmo motivo
RF-05	CSV com 4 campos	CSV com 7 campos: entram lux, acumulado, segundos sem sucesso e situação do envio
RF-06	descartar NaN do DHT11	tentar de novo e enviar com campos ausentes marcados; a falha de um sensor não descarta os outros (corrige a falha 4 da Seção 5.3 da AP1)
RNF-01	$\leq 100\,\text{mA}$ com 3 LEDs	mesmo alvo, agora com o BLE ligado e sem LEDs
RNF-02	$\leq 12\,\text{mA}$ por GPIO	**retirado**: não há mais GPIO de saída
RNF-03	aviso em $\leq 4\,\text{s}$	**retirado**: o aviso é do aplicativo; o tempo de resposta do nó virou **RF-08**
RNF-05	$\geq 8\,\text{h}$ contínuas	$\geq 24\,\text{h}$, a duração mínima para ver um ciclo diário

Fonte: Elaborada pela equipe.

2.3 Restrições

- **Componentes disponíveis:** ESP32 DevKit V1, DHT11 e BH1750 (módulo GY-302). A precisão do DHT11 (1 °C de resolução, ±2 °C de incerteza) continua sendo uma restrição aceita, e não uma escolha (**DP-1**).
- **Tensão de 3,3 V:** os dois módulos ficam no 3,3 V da placa, e não no VIN.
- **Um central por vez:** o NimBLE para de anunciar quando um central conecta. Durante a campanha, o nRF Connect precisa ficar fechado, ou o gravador não consegue conectar.
- **Sem aplicativo pronto:** o adaptador BLE do aplicativo FitoSfera ainda não existe. A campanha usa o gravador, que segue o mesmo contrato.
- **O central precisa ficar ligado e perto:** sem central não há registro de temperatura e umidade (**DP-8**). Na campanha isso significa um computador ligado, sem suspender, a no máximo cerca de 20 a 25 m do nó dentro de casa (**CT-21**).
- **Prazo:** a campanha coube entre 24/10 e 25/10, o que limita o número de ciclos diários no conjunto de dados.

2.4 Rastreabilidade

Tabela 5 – Rastreabilidade entre requisitos, implementação e testes

Requisito	Onde foi atendido	Caso de teste	Situação
RF-01	Seção 4.2; **DP-4**, **DP-9**	**CT-01**	atende
RF-02	Seção 4.5; **DP-2**	**CT-02**	atende
RF-05	Seção 4.2	**CT-05**	atende
RF-06	Seção 4.6	**CT-06**	atende
RF-07	Seção 5.3; **DP-7**	**CT-12**	atende
RF-08	Seção 4.3	**CT-13**	atende
RF-09	Seção 4.3; **DP-6**	**CT-14**	atende
RF-10	Seção 4.4	**CT-15**, **CT-16**	atende em parte
RF-11	Seção 4.3	**CT-17**	atende
RF-12	Seção 4.3; **DP-8**	**CT-20**	atende
RNF-01	Apêndice B	**CT-07**	não verificado
RNF-04	Apêndice A; **DP-5**	**CT-10**	atende
RNF-05	Seções 4.6 e 5.7	**CT-22**	atende
RNF-06	Seção 5.1; **DP-11**	**CT-21**	atende
RNF-07	Seção 5.7	**CT-22**	atende
RNF-08	Seção 5.4	**CT-18**	atende
RNF-09	Seção 5.3; **DP-7**	**CT-23**	atende

Fonte: Elaborada pela equipe.

Os vereditos vêm dos ensaios com o nRF Connect feitos antes da campanha, dos ensaios cronometrados com o gravador e da própria campanha (Seção 6.1).

3 Arquitetura do sistema

Rubrica da AP2: justificativa técnica das decisões — 25 %

3.1 Visão em camadas

A Figura 1 situa o sistema da AP2 nas quatro camadas de uma arquitetura IoT (AL-FUQAHA et al., 2015). A camada de rede, que na AP1 era o cabo USB, agora é um enlace BLE entre o nó e um central.

- **Percepção:** DHT11 (temperatura e umidade do ar) e BH1750 (luz em lux, por I^2C). Os LEDs de aviso e o módulo fotorresistor saíram.
- **Processamento na borda:** o ESP32 mede, tenta de novo quando um sensor falha, integra a luz no tempo e monta o pacote de 19 B. Ele não decide mais nada sobre a planta: a comparação com a espécie passou para o aplicativo (**DP-3**).
- **Rede:** BLE. O nó é periférico GATT; o central assina a notificação, recebe um pacote a cada 300 s e confirma cada um. Não há roteador, *broker* nem internet entre o nó e quem guarda o dado.
- **Aplicação:** na campanha, o gravador em Python, que grava o CSV, e o `qualidade.py`, que o analisa. No produto, o aplicativo FitoSfera no celular, seguindo o mesmo contrato.

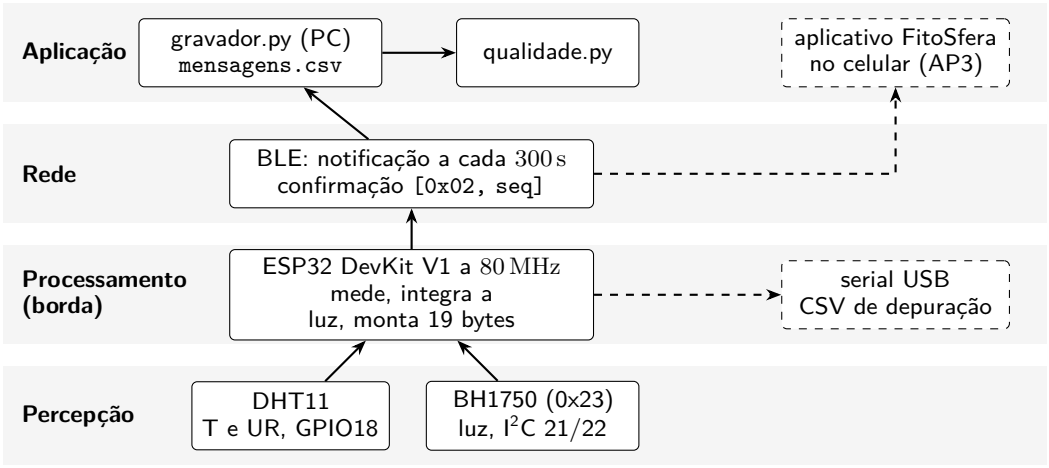


Figura 1 – Arquitetura do FitoSfera na AP2: o que rodou na campanha (contínuo) e o que fica para depois ou só serve à depuração (tracejado)

Fonte: Elaborada pela equipe.

3.2 Componentes e decisões de projeto

As caixas **DP-1** a **DP-5** vêm da AP1 e foram atualizadas para o sistema de agora. As caixas **DP-6** a **DP-10** são do Módulo 2: o que mudou na arquitetura por causa do enlace. A escolha da tecnologia em si está na **DP-11**, no Capítulo 5.

DP-1 — Sensor de temperatura e umidade do ar (mantida)

Alternativas consideradas. DHT11, DHT22 e BME280 (I^2C).

Critério de escolha. Estar disponível para a equipe, custo e resolução suficiente para perceber tendências num ambiente que muda em escala de horas.

Decisão. DHT11, como na AP1.

Evidência (número e procedência). Resolução de 1 °C e 1 % UR e incerteza de ±2 °C (AOSONG ELECTRONICS,) (FOLHA DE DADOS). Com uma leitura a cada 300 s, o intervalo mínimo de 2 s da biblioteca, que na AP1 coincidia com o período (falha 2 da Seção 5.3 da AP1), deixou de ser um risco.

Consequência assumida. A leitura continua ocupando cerca de 25 ms (Apêndice A). O BME280 é a troca prevista: entra no mesmo barramento I^2C do BH1750, sem pino novo.

DP-2 — Canal de luz: BH1750 em I^2C

Alternativas consideradas. (a) manter o fotorresistor numa entrada digital, como na AP1; (b) ler o fotorresistor pelo ADC1 (GPIO32 ou GPIO33); (c) sensor de luz calibrado em I^2C (BH1750).

Critério de escolha. O aplicativo precisa saber *quanta* luz a planta recebeu num dia, e não se o ambiente está claro ou escuro; o limiar tem de ser de software.

Decisão. (c), BH1750 (módulo GY-302) no endereço `0x23`, no modo de uma medida, com MTreg = 31.

Evidência (número e procedência). A AP1 mostrou que (a) lê uma tensão contínua numa entrada digital, numa faixa de 1,65 V sem garantia da folha de dados. (b) daria um número, mas não em lux: o fotorresistor não tem curva conhecida e o ADC do ESP32 é pouco linear nos extremos. O BH1750 entrega lux diretamente, com incerteza de ±20 % (ROHM SEMICONDUCTOR, 2011) (FOLHA DE DADOS). Na sensibilidade de fábrica (MTreg = 69) ele satura perto de 54.612 lux; com MTreg = 31 o teto sobe para cerca de 121.557 lux ($54612 \times 69/31$), o que cobre sol pleno.

Consequência assumida. Cada medida espera cerca de 100 ms de conversão com `delay()`, aceito porque acontece uma vez a cada 300 s. A resolução em luz fraca cai na mesma proporção ($69/31 \approx 2{,}2$ vezes).

DP-3 — Onde fica a decisão sobre a planta

Alternativas consideradas. (a) comparador no hardware; (b) limiares no *firmware*, com LEDs no nó (a escolha da AP1); (c) nó que só mede, com a decisão no aplicativo.

Critério de escolha. O limiar tem de ser o da espécie cadastrada, diferente para cada planta, e tem de poder mudar sem regravar o nó.

Decisão. (c). Na AP2 os três LEDs e as constantes `TEMP_ALTA` e `UMID_BAIXA` saíram do *firmware*.

Evidência (número e procedência). Na AP1 a equipe já tinha declarado que os LEDs eram de demonstração e que o limiar fixo na compilação valia para o jardim inteiro. Com o BLE, o aplicativo recebe as leituras e conhece a espécie; mandar os limiares de volta para o nó só serviria para acender um LED que ninguém olha.

Consequência assumida. Sem central, não há aviso nenhum. É o preço de (c), e é aceitável porque a planta muda em horas e o celular volta a conectar quando o usuário chega perto do jardim. Na AP3, com a IA embarcada, a decisão pode voltar em parte para o nó (Seção 8.2).

DP-4 — Temporização sem bloqueio (atualizada)

Alternativas consideradas. `delay(300000)` no fim do `loop()`, ou comparação com `millis()`.

Critério de escolha. O laço tem de atender, a qualquer momento, a assinatura da notificação, o comando de medir agora e a confirmação do central.

Decisão. Comparação com `millis()` contra um instante `proximaLeitura` (Listagem 4.1). Um pedido do central fura a fila e mede na hora.

Evidência (número e procedência). Com `delay()`, um pedido do central esperaria até 300 s; com a comparação, a resposta sai no próximo giro do laço, e **RF-08** pede no máximo 2 s. Os retornos do BLE chegam por *callbacks* do NimBLE, que só ligam uma bandeira; o trabalho é feito no `loop()`.

Consequência assumida. A única espera bloqueante que sobrou são os 100 ms da conversão do BH1750 (**DP-2**), uma vez por ciclo.

DP-5 — Escolha dos pinos (atualizada)

Alternativas consideradas. Qualquer GPIO de uso geral da placa.

Critério de escolha. Evitar os pinos que a folha de dados proíbe ou desaconselha (Espressif Systems, 2023): GPIO6 a 11 (*flash*), GPIO0, 2, 5, 12 e 15 (inicialização), GPIO1 e 3 (UART0) e GPIO34 a 39 (só entrada).

Decisão. GPIO18 (DHT11) e GPIO21/22 (SDA/SCL do BH1750).

Evidência (número e procedência). O conflito previsto na AP1 era o I^2C padrão cair em cima dos LEDs vermelho e amarelo. Com a saída dos LEDs, o par 21/22 ficou livre e o barramento usa os pinos padrão, sem `Wire.begin()` em pinos alternativos (**CT-10**).

Consequência assumida. Três pinos usados, nenhum restrito. Qualquer sensor I^2C novo entra no mesmo par, desde que o endereço seja diferente de `0x23`.

DP-6 — O que conta como entregue: confirmação da aplicação

Alternativas consideradas. (a) considerar entregue quando `notify()` volta; (b) usar indicação em vez de notificação, que a pilha BLE confirma; (c) notificação mais uma confirmação escrita pela aplicação, `[0x02, seq]`.

Critério de escolha. "Entregue" tem de significar "gravado por quem guarda o dado", e não "saiu pelo rádio".

Decisão. (c). O nó guarda a sequência do pacote em espera e só zera o contador de segundos sem sucesso quando chega a confirmação dessa sequência. O gravador grava a linha com `fsync` e só depois confirma (Listagem 4.4).

Evidência (número e procedência). Uma notificação não tem confirmação no protocolo: a volta de `notify()` só prova que o pacote foi para a fila do rádio. Uma indicação é confirmada pela pilha do central, não pelo aplicativo, que pode cair depois de receber e antes de gravar. No ensaio com o nRF Connect, sem a escrita de `02 NN`, o contador continuou crescendo de pacote em pacote (**CT-14**).

Consequência assumida. O campo `seg_sem_sucesso` de cada pacote diz há quanto tempo o nó não tem prova de entrega. Perto de 300 s, tudo certo; muito acima, a confirmação anterior se perdeu ou houve lacuna. O `qualidade.py` conta esses casos.

DP-7 — Formato do pacote: binário de 19 bytes

Alternativas consideradas. (a) JSON em texto, como o contrato de dados da disciplina; (b) estrutura binária fixa, *little-endian*.

Critério de escolha. Caber numa única notificação sem depender de negociar o MTU, e ser decodificável sem ambiguidade por um aplicativo em TypeScript.

Decisão. (b), 19 bytes (Tabela 11).

Evidência (número e procedência). Com o MTU padrão do ATT, de 23 B, cabem 20 B de dados numa notificação. O JSON equivalente, com as mesmas oito grandezas e sem espaços, tem 122 B: exigiria negociar um MTU maior, o que nem todo celular faz, ou fragmentar a mensagem. Os três pacotes de exemplo do README foram decodificados pelo decodificador em TypeScript e pelo `protocolo.py` sem divergência (**CT-12**).

Consequência assumida. O pacote não se descreve sozinho: quem recebe precisa do contrato. Uma mudança de campo exige versão nova do contrato (hoje, versão 2) e atualização do decodificador nos dois lados.

DP-8 — O que o nó guarda enquanto ninguém escuta

Alternativas consideradas. (a) nada; (b) uma fila circular de leituras na RAM, entregue quando o central volta; (c) só a luz, como integral acumulada desde que o nó ligou.

Critério de escolha. A grandeza que o aplicativo soma ao longo do dia (luz) não pode ter buraco; as que ele só compara com um limiar (temperatura e umidade) toleram.

Decisão. (c). A cada medida, o nó soma a área do trapézio entre a leitura atual e a anterior; intervalos de mais de 15 min sem leitura de luz não entram na soma.

Evidência (número e procedência). Num ensaio com o nRF Connect, depois de cerca de 55 min desconectado, o primeiro pacote trouxe de uma vez a luz do período inteiro (**CT-20**). Como o aplicativo soma diferenças entre acumulados, um pacote repetido não conta luz em dobro e um perdido não deixa buraco. Uma fila de um dia (288 leituras $\times$ 19 B $\approx$ 5,5 kB) caberia na RAM, mas exigiria um protocolo de entrega em lote que a equipe não implementou neste ciclo.

Consequência assumida. Temperatura e umidade medidas sem central se perdem. A sequência continua contando, então o conjunto de dados mostra quantas leituras faltaram, mas não os valores. É por isso que, na campanha, o gravador precisa ficar ligado o tempo todo (Seção 5.2).

DP-9 — Período de amostragem: 300 s

Alternativas consideradas. 2 s (AP1), 60 s e 300 s.

Critério de escolha. Ser bem mais rápido que a resposta da planta, que é de horas, sem gastar rádio e sem inflar o conjunto de dados.

Decisão. 300 s (`INTERVALO_MS`), com leitura extra a pedido do central.

Evidência (número e procedência). São 288 leituras e 288 $\times$ 19 B = 5.472 B por dia, doze amostras por hora. A 2 s seriam 43.200 notificações por dia, sem informação nova sobre a planta.

Consequência assumida. A luz é amostrada a cada 5 min, o que torna a integral grosseira em dias de nuvens passando. Um evento mais curto que 5 min (uma sombra passageira) pode não aparecer no conjunto de dados.

DP-10 — Quem dá a hora da leitura

Alternativas consideradas. (a) NTP no nó, o que exige WiFi; (b) relógio de tempo real externo; (c) o central carimba a hora na chegada.

Critério de escolha. Toda leitura do conjunto de dados precisa de hora real, para que a série tenha ciclo diário e para rotular eventos.

Decisão. (c). O gravador grava `ts_recebido` com o relógio do computador, sincronizado pelo sistema operacional; o aplicativo fará o mesmo com o relógio do celular.

Evidência (número e procedência). Com BLE não há rede no nó, então (a) não se aplica; (b) custaria um módulo e uma bateria para corrigir um erro que, com notificação, é do tamanho do intervalo de conexão (dezenas de milissegundos). O nó mede e notifica no mesmo giro do laço.

Consequência assumida. Nenhuma leitura sai com hora do nó: o equivalente a "`ts null`" da disciplina vale para 100 % dos pacotes, e é tratado como decisão, não como falha (**CT-19**). Uma leitura feita por `READ`, sem notificação, pode ter até 300 s de idade; o gravador não usa `READ`.

3.3 Mapa de barramento

O nó tem dois sensores e o enlace. O DHT11 continua no barramento de um fio próprio, no GPIO18, e é o dispositivo mais lento, com cerca de 25 ms por leitura. O BH1750 está sozinho no I^2C, no endereço `0x23` (pino ADDR solto), nos pinos padrão 21/22, que a saída dos LEDs liberou (**DP-5**). Na inicialização o *firmware* varre o barramento e lista os endereços na serial; se o `0x23` não responde, o nó avisa e segue sem luz. O mapa completo está no Apêndice A.

3.4 Orçamento de energia

O nó continua sempre ligado, num carregador USB de 5 V, agora com o BLE anunciando ou conectado o tempo todo e a CPU a 80 MHz, o mínimo com BLE. Sem os LEDs, o consumo estimado cai de 66 mA (AP1) para cerca de 46 mA em média, ou cerca de 1.114 mA h por dia. Quem domina é o ESP32 acordado com o rádio, que responde por cerca de 97 % da conta; os sensores ficam abaixo de 2 %. A linha do rádio e a corrente total estão detalhadas no Apêndice B e na Seção 5.5. A equipe não tem medidor USB, então esses números continuam sendo estimativa.

3.5 Lista de materiais

Tabela 6 – Lista de materiais do protótipo

Item	**Qtd.**	**Unitário (R$)**	**Total (R$)**	**Origem**
Kit eletrônica iniciante ESP32	1	180,00	180,00	Mercado Livre
Itens do kit usados no protótipo:				
Placa ESP32 DevKit V1 (ESP32-WROOM-32)	1	—	—	kit
Módulo DHT11 (3 pinos, *pull-up* na placa)	1	—	—	kit
Protoboard e fios macho-macho (9 em uso)	1	—	—	kit
Cabo USB	1	—	—	kit
Módulo BH1750 (GY-302)	1	21,04	21,04	Mercado Livre
Carregador USB 5 V, 1 A ou mais	1	—	—	da equipe
		Total	**201,04**	

Fonte: Elaborada pela equipe.

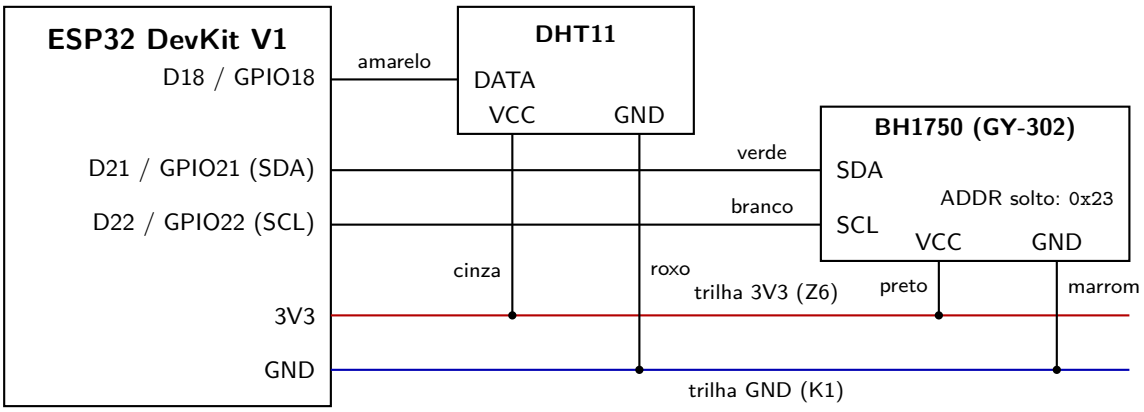
Saíram do protótipo da AP1 os três LEDs, os três resistores de 220 Ω e o módulo fotorresistor. O gravador roda num notebook Acer Aspire A515-57 que a equipe já tinha (Linux Mint 22.3, Python 3.12.3), com Bluetooth.

4 Implementação

Rubrica da AP2: funcionalidade do protótipo — 30 %

4.1 Montagem

O protótipo continua numa protoboard. O ESP32 DevKit V1 ocupa as linhas A a I, na posição mais à esquerda, com o conector USB para fora e a trilha azul (GND) para baixo. O DHT11 fica em J45 e o BH1750 em J55–59, no lugar do antigo módulo fotorresistor. Todas as ligações usam fios macho-macho. A Figura 2 mostra o esquema, e a Tabela 7 lista cada fio como está na bancada (`firmware/intrucoes_montagem.md`).



USB 5 V (carregador) → regulador da placa → 3,3 V nos pinos 3V3 e GND; fio vermelho do 3V3 à trilha Z6 e azul do GND à trilha K1

Figura 2 – Esquema elétrico do protótipo da AP2. As cores nos rótulos são as dos fios

Fonte: Elaborada pela equipe a partir de `firmware/intrucoes_montagem.md`.

Tabela 7 – Tabela de ligações

De	Para	Observação	Fio
Pino 3V3 da placa	trilha vermelha (Z6)	alimenta os dois módulos	vermelho
Pino GND da placa	trilha azul (K1)	referência comum	azul
DHT11 — DATA	D18 (GPIO18)	módulo em J45	amarelo
DHT11 — VCC	trilha vermelha	3,3 V, não VIN	cinza
DHT11 — GND	trilha azul		roxo
BH1750 — SDA	D21 (GPIO21)	módulo em J55–59	verde
BH1750 — SCL	D22 (GPIO22)		branco
BH1750 — VCC	trilha vermelha	3,3 V, não VIN	preto
BH1750 — GND	trilha azul		marrom
BH1750 — ADDR	sem ligação	endereço `0x23`	—

Fonte: Elaborada pela equipe.

Os dois módulos ficam no 3,3 V: nenhuma entrada do ESP32 tolera 5 V (Espressif Systems, 2023), e assim nem o DATA do DHT11 nem o SDA/SCL do BH1750 passam de 3,3 V. O DHT11 de três pinos traz o *pull-up* na placa, e o módulo GY-302 costuma trazer os do I²C; o barramento funcionou sem resistor externo (o `0x23` aparece na varredura). A verificação da montagem é a primeira linha da serial: `# I2C: 0x23`. Saíram da montagem da AP1 os três LEDs, os três resistores de 220 Ω, o fio laranja de aterramento dos LEDs e o módulo fotorresistor.

4.2 Estrutura do firmware

O *firmware* é C++ sobre o *framework* Arduino, compilado pelo PlatformIO no ambiente `esp32dev`. As bibliotecas são `adafruit/DHT sensor library@^1.4.6`, `claws/BH1750 @^1.3.0` e `h2zero/NimBLE-Arduino@^1.4.3` (Adafruit Industries, 2023; h2zero, 2024). Todo o código está em `firmware/src/main.cpp`, com 284 linhas; compilado, usa 36.108 B de RAM (11 %) e 616.185 B de *flash* (47 %). O nó passa pelos estados abaixo, que são os mesmos do orçamento de energia (Apêndice B):

inicialização
: uma vez, no `setup()`: reduz a CPU para 80 MHz, sorteia o id do reinício, varre o I²C, inicia o DHT11 e o BH1750 e começa a anunciar como `FitoSfera`. A primeira leitura sai 2 s depois, tempo de o DHT11 estabilizar.

aguarda
: o `loop()` trata uma confirmação pendente e volta enquanto não chega a hora da próxima leitura nem um pedido do central. O BLE continua anunciando ou conectado. Ocupa quase todo o ciclo de 300 s.

mede
: lê o DHT11 (cerca de 25 ms) e o BH1750 (cerca de 100 ms) e soma a luz na integral.

nova tentativa
: se algum sensor falhou e ainda há tentativa, agenda a próxima medida para dali a 5 s, até 3 vezes.

envia
: monta o pacote de 19 B, atualiza a característica de leitura, notifica se há central inscrito e escreve a linha CSV na serial. Volta para *aguarda*.

```
1  void loop() {
2    tratarConfirmacao();
3
4    bool pedido = pedidoDoApp;
5    if (!pedido && (long)(millis() - proximaLeitura) < 0) return;  //
        AGUARDA
6    pedidoDoApp = false;
7
8    float temp = dht.readTemperature();                            // MEDE
9    float umid = dht.readHumidity();
10   if (!luzOk && (luzOk = iniciarLuz())) Serial.println("# BH1750
        encontrado");
11   float lux = medirLuz();
12   integrarLuz(lux);
13
14   bool completo = !isnan(temp) && !isnan(umid) && !isnan(lux);
15   if (!completo && tentativas < MAX_TENTATIVAS) {                //
        NOVA TENTATIVA
```

```
    tentativas++;
    proximaLeitura = millis() + REPETICAO_MS;                    // 5 s
    return;
  }
  tentativas = 0;
  proximaLeitura = millis() + INTERVALO_MS;                      // 300 s

  bool enviado = enviarPacote(temp, umid, lux);                  // ENVIA
  // ... linha CSV na serial
}
```

Listing 4.1 – Laço principal de `firmware/src/main.cpp`: hora marcada, pedido do central e nova tentativa

Três detalhes do laço decidem o comportamento:

- a comparação `(long)(millis() - proximaLeitura) < 0` continua certa quando `millis()` volta a zero, depois de cerca de 49,7 dias;
- um pedido do central mede na hora e *reinicia* o relógio do ciclo: a próxima leitura periódica sai 300 s depois do pedido. Por isso, depois de cada reconexão, a grade de 5 min do conjunto de dados se desloca (Seção 5.7);
- o BH1750 é procurado de novo a cada ciclo, então um módulo que se solta e volta é reencontrado sem reiniciar o nó.

A linha CSV da serial tem 7 campos: `tempo_s, temp_c, umidade_pct, lux, lux_min_acum, seg_sem_sucesso, enviado`. Ela serve à depuração na bancada; o conjunto de dados é o do gravador.

4.3 Firmware de comunicação

Estados da conexão. Do lado do nó, o enlace tem três estados. *Anunciando*: sem central, o NimBLE anuncia o nome `FitoSfera` e o UUID do serviço; o nó continua medindo a cada 300 s, incrementa a sequência e acumula a luz, mas não notifica ninguém (a serial mostra `sem celular`). *Conectado*: um central conectou; o NimBLE para de anunciar, porque o nó aceita um central por vez. *Inscrito*: o central assinou a notificação da característica de leitura. A transição para *inscrito* dispara uma medida imediata, que entrega a luz acumulada enquanto o central estava fora.

```
class CallbacksDoServidor : public NimBLEServerCallbacks {
  // Um só celular por vez: o NimBLE para de anunciar ao conectar e volta ao desconectar
  void onDisconnect(NimBLEServer*) override {
    NimBLEDevice::startAdvertising(); }
};

class CallbacksDaLeitura : public NimBLECharacteristicCallbacks {
  void onSubscribe(NimBLECharacteristic*, ble_gap_conn_desc*, uint16_t subValue) override {
    if (subValue & 1) pedidoDoApp = true;
  }
};
```

Listing 4.2 – Reconexão e assinatura: quem volta a anunciar e quem dispara a medida

Reconexão. Quem reconecta é o central: o nó só volta a anunciar, no *callback* de desconexão. O gravador procura o nó por até 20 s, com espera de 2 s dobrando até 60 s entre buscas sem sucesso (Seção 4.4). Assim, depois de o nó voltar a anunciar, a reconexão leva no máximo cerca de 80 s (**RF-10**).

Confirmação. O *callback* de escrita na característica de comando só copia a sequência e liga uma bandeira; o `loop()` decide (Listagem 4.3). Uma confirmação de uma sequência que não é a do pacote em espera é ignorada, para que uma confirmação atrasada não zere o contador de um pacote que ninguém recebeu.

```
void tratarConfirmacao() {
  if (!confirmacaoRecebida) return;
  confirmacaoRecebida = false;
  if (aguardandoConfirmacao && confirmacaoSeq == seqAguardando) {
    aguardandoConfirmacao = false;
    ultimoSucessoMs = millis();      // zera os segundos sem sucesso
  }
}
```

Listing 4.3 – Só a confirmação do pacote em espera conta como sucesso

Fila local. Não há fila de leituras (**DP-8**). O que sobrevive a uma desconexão é a integral da luz (`luxSegundos`, em `double`, enviada como lux×min em `uint32`) e a contagem de sequência, que continua andando sem central. A sequência é que permite ao `qualidade.py` dizer "o nó mediu 7 vezes e ninguém recebeu", em vez de só "faltou meia hora".

Hora e NTP. O nó não tem relógio nem NTP (**DP-10**). Toda hora do conjunto de dados é a de chegada no gravador. O pacote leva o id do reinício, sorteado com `esp_random()` a cada `setup()`, para que o central saiba que a sequência e o acumulado recomeçaram.

Testamento. O BLE não tem testamento como o MQTT. O equivalente tem duas partes. Quando o nó perde a energia, o enlace cai por *timeout* de supervisão e o gravador recebe a desconexão, que anota em `eventos.csv` (**RNF-08**). Quando o nó trava com o rádio de pé, o enlace continua, mas os pacotes param: o gravador derruba a conexão depois de 630 s sem pacote (dois períodos mais 30 s) e anota `silencio`.

4.4 O gravador

A disciplina não tinha gravador BLE, então a equipe escreveu um (`gravador/gravador.py`, Python 3 com a biblioteca `bleak` (BLIDH, 2025)). Ele faz o papel do aplicativo: procura o nó, conecta, assina a notificação e, a cada pacote, grava e confirma. O decodificador está em `gravador/protocolo.py`, compartilhado com o `qualidade.py` e testado contra os três pacotes de exemplo do README (`python3 -m unittest discover gravador`, 10 testes).

```
def ao_notificar(_caracteristica, dados):
    seq = self.guardar(endereco, dados)   # escreve a linha e faz fsync
    if seq is not None:
        confirmar.put_nowait(seq)         # só então entra na fila de
            confirmação

# ... no laço da sessão
await cliente.write_gatt_char(protocolo.UUID_COMANDO,
                              protocolo.comando_confirmar(seq),
                                 response=True)
```

Listing 4.4 – O gravador grava antes de confirmar (`gravador/gravador.py`)

Três escolhas do gravador decidem o que se perde:

- ***Flush* a cada linha.** Cada linha vai para o disco com `fsync` antes de a confirmação sair. Se o computador desligar, perde-se no máximo o pacote que estava chegando, e esse pacote não foi confirmado.
- **Arquivo só de acréscimo.** Se o gravador parar e voltar, ele continua o mesmo `mensagens.csv`. Uma linha repetida (por exemplo, se o mesmo pacote chegar duas vezes) é descartada depois, pelo `qualidade.py`, que compara os 19 bytes.
- **Eventos separados das leituras.** Conexões, quedas, RSSI de cada busca e erros vão para `eventos.csv`. É dele que sai a explicação de cada lacuna.

4.5 Tratamento do sinal

O DHT11 entrega temperatura e umidade em unidades de engenharia; o *firmware* só converte para centésimos, com arredondamento, para caber em inteiros de 16 bits. O BH1750 entrega lux, já com a correção do MTreg feita pela biblioteca. Não há filtragem nem média: com uma leitura a cada 5 min, uma média móvel atrasaria a série sem ganho para uma grandeza que muda em horas. O único processamento é a integral da luz, por trapézios:

$$L_{\text{acum}} \mathrel{+}= \frac{\ell_{k-1}+\ell_k}{2}\,(t_k - t_{k-1}), \qquad \text{se } t_k - t_{k-1} \le 15\,\text{min},$$

em lux×segundo, enviada dividida por 60. Um intervalo maior que 15 min sem leitura de luz (sensor solto) fica fora da soma: o dado falta em vez de ser inventado.

4.6 Comportamento em falha

- **Um sensor devolve lixo.** O DHT11 devolve NaN quando o *checksum* falha; o BH1750 devolve valor negativo. O nó tenta de novo a cada 5 s, até 3 vezes, e então envia o que tiver, com `0x8000` na temperatura, `0xFFFF` na umidade ou `0xFFFFFFFF` na luz. A falha de um sensor não descarta os outros, o que corrige a falha 4 da AP1.
- **A alimentação cai.** O nó perde a integral da luz e a sequência. Quando volta, sorteia um id de reinício novo, a sequência recomeça em 1 e o acumulado em 0; o aplicativo e o `qualidade.py` reconhecem o reinício e não o confundem com perda.
- **O central some.** O nó volta a anunciar e continua medindo. Temperatura e umidade dessas medidas se perdem; a luz não (**DP-8**).
- **Como alguém descobre que o sistema parou.** Pelo gravador: o evento `desconectado` quando o nó cai e `silencio` quando ele trava. No produto, pelo aplicativo, que verá o campo de segundos sem sucesso crescer ou deixará de receber pacotes.

4.7 Repositório

Repositório: <https://github.com/lucasvilari/FitoSfera-Topicos>
Commit correspondente a esta entrega: **[pendente: hash do commit da entrega, marcado com git tag ap2]**

```
README.md                    visao geral, contrato BLE e validacao
firmware/                    projeto PlatformIO (abrir esta pasta)
  platformio.ini             esp32dev, DHT11, BH1750 e NimBLE
  src/main.cpp               firmware completo (284 linhas)
  intrucoes_montagem.md      ligacoes na protoboard, fio a fio
gravador/                    central BLE da campanha (escrito pela equipe)
  gravador.py                grava dados/mensagens.csv e dados/eventos.csv
  protocolo.py               contrato do pacote de 19 bytes
  qualidade.py               relatorio de qualidade
  test_*.py                  testes do decodificador e do qualidade.py
dados/                       conjunto de dados da campanha
  limpar.py, grafico.py      bruto -> limpo -> figura do Capitulo 8
docs/                        fontes LaTeX deste relatorio
```

Listing 4.5 – Estrutura do repositório na AP2

Em relação à AP1, o projeto PlatformIO mudou para a pasta `firmware/`, para separar o nó do gravador e dos dados. O *firmware* não tem credenciais: o BLE não usa senha de rede nem *broker*, então não há `secrets.h` nem `secrets.example.h`.

5 Conectividade e conjunto de dados

Rubrica da AP2: justificativa técnica das decisões — 25 %

Este capítulo documenta o Módulo 2: como o nó da AP1 passou a falar com o mundo e o que ele coletou na campanha. A equipe escolheu BLE e não WiFi. Por isso algumas peças do módulo não existem aqui (*broker*, tópicos, testamento MQTT, NTP), e cada seção diz qual é o equivalente no BLE e o que ele garante ou deixa de garantir.

5.1 Tecnologia de comunicação

Rubrica da AP2: justificativa técnica das decisões — 25 %

Tabela 8 – Matriz de decisão de conectividade

Critério	**O que o projeto exige**	**WiFi**	**BLE**	**LoRa-WAN**	**NB-IoT, LTE-M**	**Observação**
Alcance necessário	cerca de 10 m até o celular, com paredes (Capítulo 1)	**serve**	**serve**	**serve**	**serve**	BLE medido: 20 a 25 m com várias paredes; 45 a 50 m com uma (**CT-21**)
Energia disponível	tomada USB; cerca de 1.114 mA h/dia (Apêndice B)	**serve**	**serve**	**serve**	**serve**	na tomada nada restringe; na bateria, o pico de TX do WiFi (240 mA) é quase o dobro do BLE (130 mA)
Dado por transmissão	19 B a cada 300 s (5.472 B/dia)	**serve**	**serve**	**com ressalva**	**serve**	LoRaWAN: 288 mensagens/dia cabem em 30 s de ar em SF7 (20,7 s), não em SF12 (521 s)
Latência aceitável	minutos; a planta responde em horas	**serve**	**serve**	**serve**	**serve**	nenhuma restringe
Infraestrutura no local	o celular de quem cuida do jardim; a casa tem roteador WiFi	**com ressalva**	**serve**	**não serve**	**não serve**	WiFi exige gravar a senha da rede no nó; LoRa exige *gateway*; NB-IoT exige chip e plano

Fonte: Elaborada pela equipe. Correntes de pico de Espressif Systems (2023); tempo no ar de LoRaWAN calculado para 32 B no ar (19 de dados e 13 de cabeçalho), 125 kHz, CR 4/5.

Tabela 9 – Marcas que mudaram desde a AP1

Tecnologia	**Na AP1 (19/09)**	**Agora**	**O que mudou, com o número**
BLE	não avaliada	escolhida	O contrato BLE fechou o pacote em 19 B, que cabe numa notificação com o MTU padrão (**DP-7**). Os ensaios com o nRF Connect mostraram que o central é o celular, e que a luz acumulada atravessa uma desconexão de 55 min sem perda (**DP-8**). O alcance medido, de 20 a 25 m com paredes, é o dobro do que o jardim exige.
WiFi	candidata (AP1, Seção 7.2)	**com ressalva**	Na AP1 a equipe apontou WiFi porque o ESP32 já o tem. O produto, porém, é um aplicativo de celular: com WiFi o nó precisaria da senha da rede e de um servidor para chegar ao celular, peças que o BLE dispensa.
LoRaWAN	não avaliada	**não serve**	A conta do tempo no ar mostra que o volume cabe em SF7, mas o kit não tem módulo LoRa e não há *gateway* perto; o alcance de quilômetros não é exigido por um jardim doméstico.

Fonte: Elaborada pela equipe.

DP-11 — Tecnologia de comunicação

Alternativas consideradas. WiFi, BLE, LoRa/LoRaWAN e NB-IoT/LTE-M.

Critério de escolha. Distância entre o nó e o primeiro equipamento que recebe o dado; orçamento de energia do Apêndice B; bytes e frequência das mensagens; infraestrutura existente no local.

Decisão. BLE, com o nó como periférico GATT e o celular (no produto) ou o gravador (na campanha) como central.

Evidência (número e procedência). O jardim exige cerca de 10 m até o celular, e o enlace alcançou 20 a 25 m dentro de casa, com várias paredes, e 45 a 50 m com uma parede só (**CT-21**). O nó entrega 19 B a cada 300 s; na campanha, a completude foi de 97,6 % (Seção 5.7). O celular já está na casa do usuário: o BLE não exige roteador, senha de rede nem servidor, e o pico de transmissão é de 130 mA, contra 240 mA do WiFi (Espressif Systems, 2023) (FOLHA DE DADOS).

Consequência assumida. O dado só sai do nó quando há um central por perto e ligado. Sem ele, temperatura e umidade se perdem (**DP-8**); não há como consultar o jardim de longe. A escolha deixa de valer se o usuário quiser acompanhar as plantas fora de casa, ou se o nó for para um canteiro a mais de cerca de 20 m de onde o celular fica, atrás de paredes: aí o WiFi, com o roteador que a casa já tem, passa a ser a tecnologia certa, com o custo de provisionar a senha da rede no nó.

5.2 Caminho do dado, do sensor ao arquivo

Rubrica da AP2: funcionalidade do protótipo — 30 %

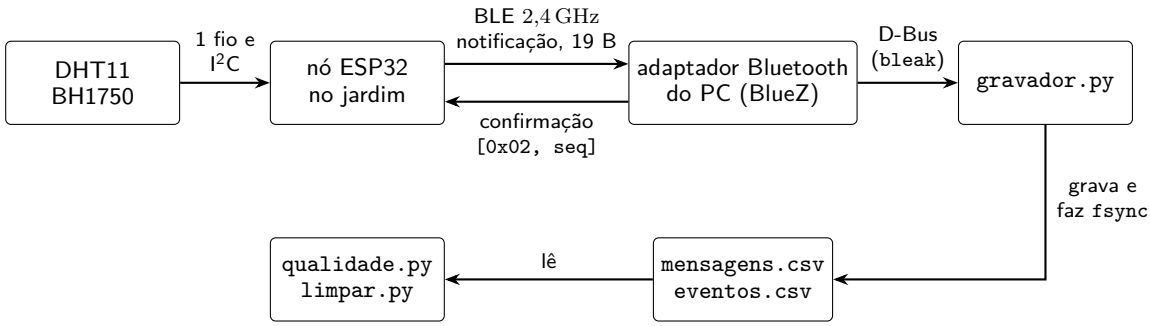


Figura 3 – Caminho do dado durante a campanha de coleta. A seta de volta é a confirmação, escrita depois de gravar

Fonte: Elaborada pela equipe.

Tabela 10 – Onde roda cada peça, e o que acontece se ela parar

Peça	**Onde rodou**	**Alimentação**	**Se parar, o que se perde**
Nó ESP32	num quarto, junto a uma janela aberta, com sol direto em parte do dia, a 50 cm do chão	carregador USB 5 V na tomada	tudo o que ele deixaria de medir, e a luz acumulada até ali; o reinício aparece no id do reinício
Enlace BLE	entre o nó e o PC, a cerca de 5 m	—	se cai e volta, a assinatura faz o nó medir na hora; temperatura e umidade do intervalo se perdem, a luz não; a sequência mostra quantas leituras faltaram
Gravador	notebook Acer Aspire A515-57 com Linux Mint 22.3 (Cinnamon), dentro da casa	tomada	o mesmo que o enlace: sem *broker* não há quem guarde no meio do caminho. Quem guarda é quem recebe
Arquivo	disco do mesmo computador	—	cada linha vai para o disco antes da confirmação; um desligamento perde no máximo o pacote que estava chegando, que não foi confirmado

Fonte: Elaborada pela equipe.

A frase do Lab 03, "o *broker* transporta; quem guarda é quem assina", no BLE fica mais curta: não há quem transporte. O gravador é ao mesmo tempo o assinante e o único lugar onde o dado existe depois de sair do nó. Por isso ele confirma só depois do `fsync` (**DP-6**), e por isso, durante a campanha, o computador não pode suspender nem ficar longe do nó.

5.3 Contrato de dados

Rubrica da AP2: justificativa técnica das decisões — 25 %

O contrato é o pacote binário de 19 B da versão 2 do contrato BLE, *little-endian*, notificado na característica de leitura. As quatro chaves da disciplina (`id`, `boot`, `seq`, `ts`) têm equivalente, mas não todas dentro do pacote: o `id` é o endereço BLE do nó, que o gravador anota, e o `ts` é a hora de chegada, porque o nó não tem relógio (**DP-10**).

Tabela 11 – Contrato de dados da telemetria (versão 2)

Campo	Bytes	Tipo	Unidade	Exemplo	Para que serve
Fora do pacote, anotados pelo gravador					
`endereco`	—	texto	—	`8C:94:DF:8F:FA:16`	o `id` da disciplina: identifica o nó
`ts_recebido`	—	real	s (época Unix)	1792861964	o `ts`: hora de chegada no gravador
No pacote					
`seq`	0	`uint8`	—	7	conta os pacotes; o salto denuncia a perda, a repetição a duplicata; volta a 0 depois de 255
`id_reinicio`	1–2	`uint16`	—	`0xA1B2`	o `boot`: sorteado a cada ligamento; separa um reinício de uma perda
`lux_min_acum`	3–6	`uint32`	lux·min	150.000	luz desde que o nó ligou; a diferença entre dois pacotes é a luz do intervalo
`seg_sem_sucesso`	7–10	`uint32`	s	1.830	tempo desde a última confirmação que o nó recebeu
`temp_c`	11–12	`int16`	0,01 °C	2.514	temperatura; `0x8000` = sem leitura
`umid_pct`	13–14	`uint16`	0,01 %	5.800	umidade relativa; `0xFFFF` = sem leitura
`lux`	15–18	`uint32`	lux	20.000	luz no instante da medida; `0xFFFFFFFF` = sem leitura

Fonte: Elaborada pela equipe, a partir do contrato BLE do README (“Contrato BLE (versão 2)”).

A unidade não está no nome da chave dentro do pacote, porque o pacote não tem nomes: ela está no contrato e, no CSV do gravador, no nome da coluna (`temp_c`, `umid_pct`). O pacote de exemplo da tabela é o primeiro dos três do README, decodificado pelo `protocolo.py`.

```
ts_recebido,hora_utc,endereco,tamanho,bytes_hex,seq,id_reinicio,
    lux_min_acum,seg_sem_sucesso,temp_c,umid_pct,lux
1792861964.405,2026-10-24T17:12:44.405+00:00,8C:94:DF:8F:FA:16,19,16
    eaebee4005002c010000cc10540b710f0000
    ,22,60394,344302,300,43.0,29.0,3953
```

Listing 5.1 – Uma mensagem real de telemetria, copiada do `mensagens.csv`

Contrato GATT. O nó anuncia o nome `FitoSfera` e um serviço com duas características (Tabela 12). A mudança da versão 1 para a 2 foi a confirmação da aplicação e o campo de segundos sem sucesso: na versão 1 o “enviado” da pilha BLE contava como entrega.

5.4 Contrato GATT e garantias de entrega

Rubrica da AP2: justificativa técnica das decisões — 25 %

No BLE não há tópicos nem QoS. O papel do contrato de tópicos é feito pelo contrato GATT: quais características existem, em que direção o dado anda e quem garante o quê.

Tabela 12 – Contrato GATT do nó (o equivalente do contrato de tópicos MQTT)

UUID	**Direção**	**Proprie-dades**	**Quem usa**	**Por quê**
`f1705f3a-0001-...` (serviço)	—	—	—	UUID próprio de 128 bits, anunciado: o gravador acha o nó pelo nome ou pelo serviço
`f1705f3a-0002-...` (leitura)	nó → central	READ, NOTIFY	gravador assina	notificação, e não indicação: a garantia de entrega é a confirmação da aplicação (**DP-6**); antes do primeiro envio tem 0 bytes, para não mostrar leitura velha como atual
`f1705f3a-0003-...` (comando)	central → nó	WRITE, WRITE sem resposta	gravador escreve	`01` mede agora; `02 NN` confirma o pacote NN. O gravador escreve com resposta, para saber se a confirmação chegou

Fonte: Elaborada pela equipe, a partir do contrato BLE do README. Os UUIDs completos terminam em `-4c7e-9a5b-3d0a6e2b9c10`.

1. Sessão persistente ou limpa, no nó e no gravador.
Não há sessão: o nó não faz *bonding* e não guarda nada sobre o central. Cada conexão começa do zero, e é a assinatura que dispara a entrega do que se acumulou. O equivalente ao *client ID* duplicado é ter dois centrais: o nó aceita um por vez, então com o nRF Connect conectado o gravador não acha o nó. A regra da campanha foi deixar só o gravador.

2. O que o nó faz com as leituras quando o central está fora de alcance.
Não há fila (**DP-8**). O nó continua medindo a cada 300 s, a sequência continua andando e a luz continua sendo somada. Na volta, o primeiro pacote traz a luz do período inteiro; as temperaturas e umidades do período se perderam, e a sequência diz quantas foram. Uma fila de um dia caberia em 5,5 kB de RAM e é a melhoria mais barata para a AP3.

3. Keepalive e testamento.
O keepalive do BLE é o *timeout* de supervisão da conexão, escolhido pelo central: se nenhum pacote do enlace chega nesse tempo, os dois lados consideram a conexão perdida. No gravador isso vira o evento `desconectado`, que é o "`offline`" do testamento. Medido na campanha: 3,6 e 5,9 s entre tirar a energia do nó e o evento `desconectado` (**CT-18**). Como o enlace pode continuar de pé com o *firmware* travado, o gravador tem um segundo vigia: 630 s sem pacote derrubam a conexão. O laço do nó nunca fica parado mais que cerca de 125 ms (DHT11 mais BH1750), muito abaixo de qualquer *timeout* de supervisão.

4. Segurança mínima.
Nenhuma neste ciclo, e isso é uma limitação declarada (Capítulo 8). Qualquer central ao alcance pode conectar, ler as leituras e escrever na característica de comando, inclusive confirmar um pacote que não gravou. Não há credencial no repositório porque não há credencial: nada de `secrets.h`. O caminho para a AP3 é pareamento com *bonding* e permissões de escrita só para o central pareado.

5.5 Custo real da comunicação

Rubrica da AP2: justificativa técnica das decisões — 25 %

Na AP1 o rádio estava desligado, e a linha do WiFi entrou como estimativa (pico de cerca de 310 mA). Na AP2 o rádio é BLE e fica ligado o tempo todo, anunciando ou conectado. Os tempos abaixo saem do relógio do gravador, pelo `eventos.csv`, e são calculados pelo `qualidade.py`. A equipe não tem medidor USB, então a corrente vem da folha de dados (pico) e da estimativa do README (média).

Tabela 13 – Linha do rádio: estimada na AP1 *versus* medida na AP2

Etapa	**Estimado (AP1)**	**Medido (AP2)**	**Como foi medido**	**Origem da corrente**
Busca até achar o nó	—	0,7 s	`nao_encontrado` e `encontrado`	—
Conexão (achado → conectado)	—	1,1 s	`qualidade.py`, enlace BLE	—
Conexão → primeiro pacote	—	0,3 s	idem	—
Reconexão depois de queda	—	20,1 s	`desconectado` → `conectado`	—
Transmissão (pico)	310 mA (WiFi)	130 mA (BLE)	—	FOLHA DE DADOS
Conectado e ocioso (média)	—	45 mA	não medido	ESTIMATIVA (README)
Rádio por dia	—	1.080 mA h	Apêndice B	

Fonte: Elaborada pela equipe. Corrente de pico de Espressif Systems (2023).

No BLE o custo não está na transmissão, que dura frações de milissegundo por pacote de 19 B, mas em manter o rádio e a CPU acordados para anunciar e manter a conexão. Com o nó na tomada isso não restringe; com bateria, seria a linha que dimensiona o sistema (Apêndice B).

5.6 A campanha de coleta

Rubrica da AP2: robustez, testes e documentação — 25 %

Tabela 14 – Ficha da campanha de coleta

Item	**Valor**
Local de instalação do nó	num quarto, junto a uma janela aberta, com sol direto em parte do dia, a 50 cm do chão
Início e fim (data e hora)	24/10/2026 14:07 a 25/10/2026 14:13 (primeira leitura às 14:07:44, última às 14:12:32)
Duração total	24,1 h
Período de amostragem	300 s
Leituras esperadas (duração ÷ período)	290
Número de nós	1
Onde rodou o gravador	notebook Acer Aspire A515-57, Linux Mint 22.3 (Cinnamon), Python 3.12.3, na tomada
Como a equipe acompanhou a coleta	saída do `gravador.py` no terminal e `qualidade.py` rodado durante os ensaios e algumas vezes por dia; diário de eventos anotado pela equipe
Coleta real ou simulada	real
Por que esta duração basta para a AP3	temperatura e luz têm ciclo diário: com menos de 24 h, o conjunto não mostra um dia inteiro. As 24,1 h cobrem 1 ciclo diário completo

Fonte: Elaborada pela equipe.

As leituras esperadas saem da conta, e não do arquivo: o nó deveria entregar uma leitura a cada 300 s durante toda a duração declarada. Leituras extras, disparadas pela assinatura a cada reconexão, não entram como esperadas; o `qualidade.py` as conta à parte.

5.7 Qualidade do conjunto de dados

Rubrica da AP2: robustez, testes e documentação — 25 %

Os números saem do `qualidade.py` rodado sobre o arquivo final da campanha, e a saída completa está no Apêndice F. Como cada reconexão reinicia o relógio do ciclo (Seção 4.2), as leituras não caem numa grade fixa; por isso a completude é calculada por janelas: a duração é dividida em janelas de 300 s, e conta-se quantas têm pelo menos uma leitura única,

$$\text{completude}\,[\%] = \frac{\text{janelas de } 300\,\text{s com leitura}}{\text{leituras esperadas}} \times 100.$$

A razão entre leituras únicas do ciclo e leituras esperadas, que é a fórmula da disciplina, também é impressa, e as duas ficam próximas quando não há muitas reconexões.

Tabela 15 – Qualidade do conjunto de dados coletado

Métrica	**Valor**	**O que explica o valor**
Leituras esperadas	290	Tabela 14
Leituras recebidas (únicas)	288	287 do ciclo e 1 extra; faltam as 7 da lacuna do **CT-16**, compensadas em parte pelas leituras de reconexão
extras, fora do ciclo	1	a assinatura reinicia o ciclo, então a leitura da reconexão quase sempre toma o lugar da do ciclo; só conta como extra quando chega colada a outra
Faltam (a sequência pulou)	12	o nó mediu e ninguém recebeu: 7 no **CT-16** (gravador parado), 1 no **CT-15** (Bluetooth desligado) e 4 a seq 1 de reinícios, medida 2 s após ligar, antes da reconexão
Repetidas (descartadas)	0	nenhum pacote chegou duas vezes
Reinícios do nó	5	todos provocados: 4 no **CT-18** (USB arrancado) e 1 no **CT-17** (botão EN)
Leituras sem hora do nó	todas	o nó não tem relógio; a hora é a de chegada (**DP-10**)
Segundos sem confirmação (mediana · máximo)	300 · 2.138 s	o máximo é o primeiro pacote depois do **CT-16**; perto de 300 s, a confirmação anterior chegou
Campos ausentes (T · UR · luz)	0 · 0 · 0	falhas de sensor depois das 3 tentativas
Mensagens fora do contrato	0	tamanho diferente de 19 B
Completude	**97,6 %**	283 de 290 janelas com leitura; as 7 vazias são todas da lacuna do **CT-16**. Pela fórmula da disciplina, 287 de 290 (99,0 %)

Fonte: Saída do `qualidade.py` sobre `dados/mensagens.csv`.

Tabela 16 – Lacunas do conjunto de dados

Início	**Fim**	**Duração**	**Leituras perdidas**	**Causa**	**Tratamento**
24/10 15:30:42	24/10 16:06:21	35 min 39 s	7	**CT-16**: gravador parado de propósito por 35 min; a luz do intervalo chegou no pacote seguinte	mantida como lacuna

Fonte: Elaborada pela equipe, a partir da seção “Lacunas” do `qualidade.py` e do diário do Apêndice F.

O `qualidade.py` já sugere a causa de cada lacuna pelo que o pacote seguinte traz: se o id do reinício mudou, o nó reiniciou; se a sequência pulou, o nó mediu e o enlace ou o gravador falharam; se nenhum dos dois, o nó deixou de medir. O arquivo de eventos completa a explicação (gravador parado, desconexão).

5.8 Descrição do conjunto de dados

Rubrica da AP2: robustez, testes e documentação — 25 %

O arquivo final é o limpo, `dados/mensagens_limpo.csv`, gerado do bruto do gravador por `dados/limpar.py`. A limpeza tira as linhas fora do contrato e as repetidas, ordena por chegada e acrescenta colunas calculadas; nenhum valor é interpolado. O bruto (`dados/mensagens.csv`) e os eventos (`dados/eventos.csv`) ficam no repositório ao lado dele.

Tabela 17 – Descrição do conjunto de dados (`dados/mensagens_limpo.csv`)

Campo	**Tipo**	**Unidade**	**Faixa observada**	**Observação**
`ts`	real	s (época Unix)	1792861664 a 1792948352	hora de chegada no gravador, relógio do PC
`hora_local`	texto	—	24/10 14:07:44 a 25/10 14:12:32	a mesma hora em Teresina (UTC−3)
`id_reinicio`	texto (hex)	—	6 valores	muda a cada ligamento do nó
`seq`	inteiro	—	0 a 255	volta a 0 depois de 255
`temp_c`	real	°C	22,6 a 44,4	vazio = sem leitura; incerteza ±2 °C
`umid_pct`	real	%	26 a 63	vazio = sem leitura; passos de 1 %
`lux`	inteiro	lux	0 a 61 571	vazio = sem leitura; incerteza ±20 %
`lux_min_acum`	inteiro	lux·min	0 a 1 245 993	recomeça em 0 a cada reinício
`luz_nova_lux_min`	inteiro	lux·min	0 a 267 969	diferença do acumulado; é o que se soma para a luz do dia
`seg_sem_sucesso`	inteiro	s	2 a 2138	desde a última confirmação recebida pelo nó
`reinicio`	0/1	—	—	1 na primeira leitura de um reinício
`perdidas_antes`	inteiro	leituras	0 a 7	medidas que o nó fez e ninguém recebeu antes desta
`extra`	0/1	—	—	1 se chegou a menos de meio período da anterior
`lacuna_antes_s`	inteiro	s	2139 (1 valor)	preenchido quando o intervalo passou de 450 s

Fonte: Elaborada pela equipe.

Arquivo: `dados/mensagens_limpo.csv` Tamanho: 22,6 kB Linhas: 288 leituras (mais o cabeçalho) *Commit*: `[pendente: hash]`

O que a equipe quer que um modelo aprenda com estes dados? A classificar a condição do ambiente do jardim como um todo, sem uma planta específica. É um problema de classificação: a cada leitura (ou janela de leituras), o modelo diz se o ambiente está adequado para um jardim em geral ou em que condição de estresse está (calor, frio, ar seco, ar úmido, pouca luz, sol forte). Os rótulos vêm de faixas gerais de cultivo da literatura e do registro de eventos (Apêndice F); a hora do dia e a luz acumulada (com ela, a luz integrada do dia em mol/m^2, DLI) entram como atributos, porque a mesma leitura pode ser normal ao meio-dia e anormal à noite. A comparação planta a planta não é papel do modelo: fica

com o aplicativo, que cruza a condição do ambiente com os valores recomendados de cada planta, guardados num banco de dados (**DP-3**).

O conjunto tem exemplos suficientes disso? Para treinar, ainda não. O que conta é o número de dias completos, porque a luz do dia só se mede com um dia inteiro: a campanha teve uma janela contínua de 24,1 h com 97,6 % de completude, mas nenhum dia de calendário com mais de 90 % das 288 janelas, porque começou e terminou no meio da tarde (117 leituras em 24/10 e 171 em 25/10). Há exemplos de sol direto (até 61.571 lux), de calor (até 44,4 °C, com o sensor ao sol), de noite escura e de resfriamento pelo ar-condicionado (até 22,6 °C), mas cada um aparece uma vez só, e não há frio nem ar úmido de verdade (umidade máxima de 63 %). O conjunto serve para validar o caminho do dado e os atributos; a AP3 precisa de vários dias seguidos, de preferência com o nó também fora do quarto. As mudanças bruscas (sol direto, sombra, janela aberta) estão anotadas no registro de eventos do Apêndice F; sem esse registro, a rotulagem de novembro vira adivinhação.

6 Verificação e testes

Rubrica da AP2: robustez, testes e documentação — 25 %

Cada requisito do Capítulo 2 tem um caso de teste. Os procedimentos passo a passo estão no Apêndice C. Os casos herdados da AP1 mantêm o número; os de LEDs (**CT-03**, **CT-04**, **CT-08** e **CT-09**) saíram junto com os LEDs. Há três fontes de evidência: os ensaios com o nRF Connect num celular Android, feitos antes da campanha e registrados no README; os testes automáticos do gravador (`python3 -m unittest discover gravador`, 10 testes, todos passando); e o `mensagens.csv` e o `eventos.csv` da campanha, lidos pelo `qualidade.py`.

6.1 Casos de teste

Tabela 18 – Casos de teste e resultados

ID	Requisito	O que se verifica	Resultado obtido	Veredito
CT-01	RF-01	intervalo entre leituras do ciclo, na campanha	mediana 300,2 s; máximo 2.139 s, na lacuna do **CT-16**	atende
CT-02	RF-02	leitura válida sob sol direto	70 619, 70 682 e 70 530 lux em 3 pedidos (24/10, 14h06), sem saturar	atende
CT-05	RF-05	10 linhas CSV da serial com 7 campos	11 de 11 linhas com 7 campos, depois do cabeçalho (`ct05.py`)	atende
CT-06	RF-06	fio do DHT11 solto e reconectado	3 tentativas a cada 5 s e envio com campos ausentes; volta sozinho (nRF Connect)	atende
CT-07	RNF-01	corrente média na USB	não medida: a equipe não tem medidor USB; estimativa de 46 mA	não verificado
CT-10	RNF-04	pinos contra a folha de dados	GPIO18, 21 e 22; nenhum restrito	atende
CT-12	RF-07	pacote no contrato	3 de 3 exemplos do README decodificados (teste automático); pacotes de 19 B no nRF Connect; teste de fumaça e ensaios com o gravador: 19 de 19 pacotes com 19 B; na campanha, 0 fora do contrato em 288	atende
CT-13	RF-08	medir agora e assinatura	assinatura: pacote em 0,47 s; 10 de 10 `01` com pacote novo e sequência seguinte, latência mediana 0,17 s, máxima 0,22 s (`ensaio.py`)	atende
CT-14	RF-09	confirmação zera o contador	sem confirmação, 114, 117 e 121 s; com `02` de sequência errada, 124 s; com a do pacote em espera, 3 s no pacote seguinte (`ensaio.py`)	atende
CT-15	RF-10	**queda do enlace** (Bluetooth do PC desligado 2 min)	reconectou 7,2 s depois do `power on`; 1 leitura perdida, medida com o rádio desligado	atende
CT-16	RF-10	**o gravador para** por alguns minutos	35 min parado: lacuna de 35 min 39 s, 7 leituras perdidas pela sequência; o primeiro pacote da volta trouxe a luz do intervalo	atende em parte
CT-17	RF-11	**o nó reinicia** (botão EN)	id novo; 1 reinício, 0 repetidas e 0 perdas falsas; a seq 1 saiu antes da reconexão e conta como perda real	atende
CT-18	RNF-08	**o nó morre sem avisar** (USB arrancado)	3,6 e 5,9 s (2 medidas confiáveis de 4; nenhuma acima de 10 s)	atende
CT-19	DP-10	**antes do NTP**: hora das primeiras leituras	todas as leituras sem hora do nó; hora do gravador em todas (288 de 288); 1º pacote 0,1 a 0,6 s depois do `conectado`	atende
CT-20	RF-12	luz através de uma desconexão	depois de cerca de 55 min fora, o primeiro pacote trouxe a luz do período inteiro (nRF Connect); com o gravador, 16.240 lux · min em 35,6 min parado (**CT-16**)	atende
CT-21	RNF-06	alcance máximo do enlace	20 a 25 m dentro de casa, com várias paredes; 45 a 50 m com uma parede	atende
CT-22	RNF-05, RNF-07	**longa duração**: a campanha	24,1 h; completude 97,6 %; 0 campos ausentes; 1 lacuna, explicada	atende
CT-23	RNF-09	tamanho do pacote	`sizeof(pacote)` = 19; 19 bytes no nRF Connect e em cada	atende

6.2 Testes de robustez: os testes mínimos da AP2

Os seis testes mínimos da disciplina foram pensados para WiFi e MQTT. Abaixo, cada um com o equivalente no BLE e o que o resultado mostra.

a) **Queda do enlace** (**CT-15**; no WiFi, desligar o roteador). Com o gravador conectado, desligar o Bluetooth do computador por 2 min (`bluetoothctl power off` e `power on`). O esperado é o evento `desconectado`, buscas sem sucesso e, depois de religar, `encontrado`, `conectado` e um pacote extra imediato. A "fila local" não existe (**DP-8**): o que o nó segura é a luz acumulada e a contagem de sequência. Resultado: `desconectado` 3 s depois do `power off`, cinco tentativas com o adaptador desligado e, depois do `power on`, `encontrado` em 6,2 s, `conectado` em 7,2 s e o pacote extra logo em seguida. A sequência pulou de 22 para 24: o `power off` caiu 297 s depois da última leitura, e o ciclo seguinte aconteceu com o rádio desligado. O pacote da volta trouxe `seg_sem_sucesso` = 424 s, coerente com a última confirmação.

b) **O nó morre sem avisar** (**CT-18**). Arrancar o cabo USB com o gravador conectado. O esperado é `desconectado` em poucos segundos, pelo *timeout* de supervisão. Resultado: 3,6 e 5,9 s nas duas medidas confiáveis; nas outras duas, o Enter que marcava o arranque saiu atrasado (numa delas, depois do próprio evento). O gravador reconectou sozinho nas quatro vezes.

c) **O nó reinicia** (**CT-17**). Apertar EN. O esperado é um id de reinício novo, sequência 1 e acumulado 0, e o `qualidade.py` contar um reinício, nenhuma repetida e nenhuma perda falsa (a regra é que, depois de um reinício, só faltam as sequências anteriores à primeira recebida). Resultado: id novo (`6d2e`), `desconectado` 1,9 s e `conectado` 5,6 s depois do EN; o `qualidade.py` contou o reinício, nenhuma repetida e nenhuma perda falsa. A primeira sequência recebida foi a 2, com acumulado de 31 lux·min: a seq 1, medida 2 s depois de ligar, saiu antes de o gravador reconectar e se perdeu de verdade. Aconteceu em 4 dos 5 reinícios da campanha.

d) **O gravador para** (**CT-16**). Parar o gravador com Ctrl+C por 35 minutos e rodar de novo. No MQTT, uma sessão persistente faria o *broker* guardar o atraso; no BLE não há quem guarde. O esperado, por projeto, é uma lacuna no CSV, a sequência mostrando as leituras que o nó fez sem ninguém ouvindo e o primeiro pacote da volta trazendo a luz do intervalo. É um teste que *falha* contra o critério da disciplina ("sem buraco no CSV"), e a falha é a consequência assumida na **DP-8**. Resultado: lacuna de 35 min 39 s, com 7 leituras que o nó fez sem ninguém ouvindo (seq 4 a 10); o primeiro pacote da volta trouxe 16.240 lux · min novos em 35,6 min, média de 456 lux, coerente com os 569 e 419 lux medidos antes e depois. A primeira tentativa, com o gravador num script, falhou: o `kill -INT` foi ignorado (ver "O que falhou").

e) **Antes do NTP** (**CT-19**). O nó não tem NTP: nenhuma leitura sai com hora do nó, nem antes nem depois de ligar. A equipe decidiu que a hora é a de chegada no gravador (**DP-10**). O que se confere é que a primeira leitura depois de ligar o nó chega cerca de 2 s depois da conexão e tem hora do gravador como as outras. Resultado: nas 288 leituras, só a hora do gravador; depois de cada reinício, o primeiro pacote chegou de 0,1 a 0,6 s depois do `conectado`, porque a assinatura dispara a medida.

f) **Longa duração** (**CT-22**). A própria campanha (Seções 5.6 e 5.7).

6.3 O que falhou

1. **Gravação a 460.800 Bd falhava** com o cabo e a porta usados. A velocidade de gravação foi reduzida para 115.200 Bd no `platformio.ini`.
2. **Um carregador rápido desligava o nó.** Com a carga baixa do protótipo (cerca de 46 mA), um carregador rápido religava a saída de tempos em tempos, e cada religamento era um reinício. Foi trocado por um carregador USB comum de 5 V. É o tipo de falha que o id do reinício torna visível no conjunto de dados.
3. **O nó não segura temperatura e umidade sem central.** Não é defeito de código: é a consequência da **DP-8**, e o **CT-16** existe para medi-la.
4. **Na campanha, o gravador ignorou o Ctrl+C.** Às 14:34 de 24/10, o **CT-16** automático mandou `kill -INT` ao gravador, que rodava em segundo plano num script; num script sem controle de tarefas, esses processos nascem com o SIGINT ignorado, e o gravador seguiu gravando, sem lacuna. A equipe corrigiu o `gravador.py` (SIGINT e SIGTERM encerram como o Ctrl+C) e refez o ensaio às 15:31.
5. **O BlueZ segurou a conexão de um gravador morto.** Na troca do processo, às 15:11, o gravador antigo foi encerrado à força e o BlueZ manteve a conexão aberta; o nó, ainda conectado, parou de anunciar, e o gravador novo não o achava. Resolvido com `bluetoothctl disconnect` (55 s sem leitura, sem perda). Com o encerramento correto, o problema não voltou.
6. **A primeira medida depois de ligar se perde.** O nó mede 2 s depois de ligar, e o gravador leva de 5 a 28 s para reconectar: a seq 1 se perdeu em 4 dos 5 reinícios. Guardar o último pacote não confirmado e reenviá-lo na assinatura resolve.
7. **O sol aquece o DHT11.** Com o nó ao sol direto, a temperatura subiu de 35,2 para 43,0 °C em 2 min e chegou a 44,4 °C: é o sensor aquecido, não o ar do quarto. As leituras ao sol direto (até cerca de 14:55 de 24/10) precisam dessa ressalva na rotulagem.

As quatro falhas que a AP1 encontrou na revisão foram resolvidas na AP2: a luz numa entrada digital saiu com o BH1750 (**DP-2**); o período igual ao intervalo mínimo do DHT11 deixou de ser risco com 300 s (**DP-9**); o I^2C deixou de colidir com os LEDs porque os LEDs saíram (**DP-5**); e a falha do DHT11 deixou de descartar a luz (Seção 4.6).

7 Resultados

Rubrica da AP2: comunicação dos resultados — 20 %

7.1 O sistema em operação

A Figura 4 mostra a campanha inteira: temperatura, umidade e luz, com o mesmo eixo de tempo. A linha é interrompida em toda lacuna maior que 450 s (1,5 período), sem interpolação; as faixas cinzas marcam as lacunas e as linhas tracejadas vermelhas, os reinícios do nó. A figura é gerada por `dados/grafico.py` a partir de `dados/mensagens_limpo.csv`.

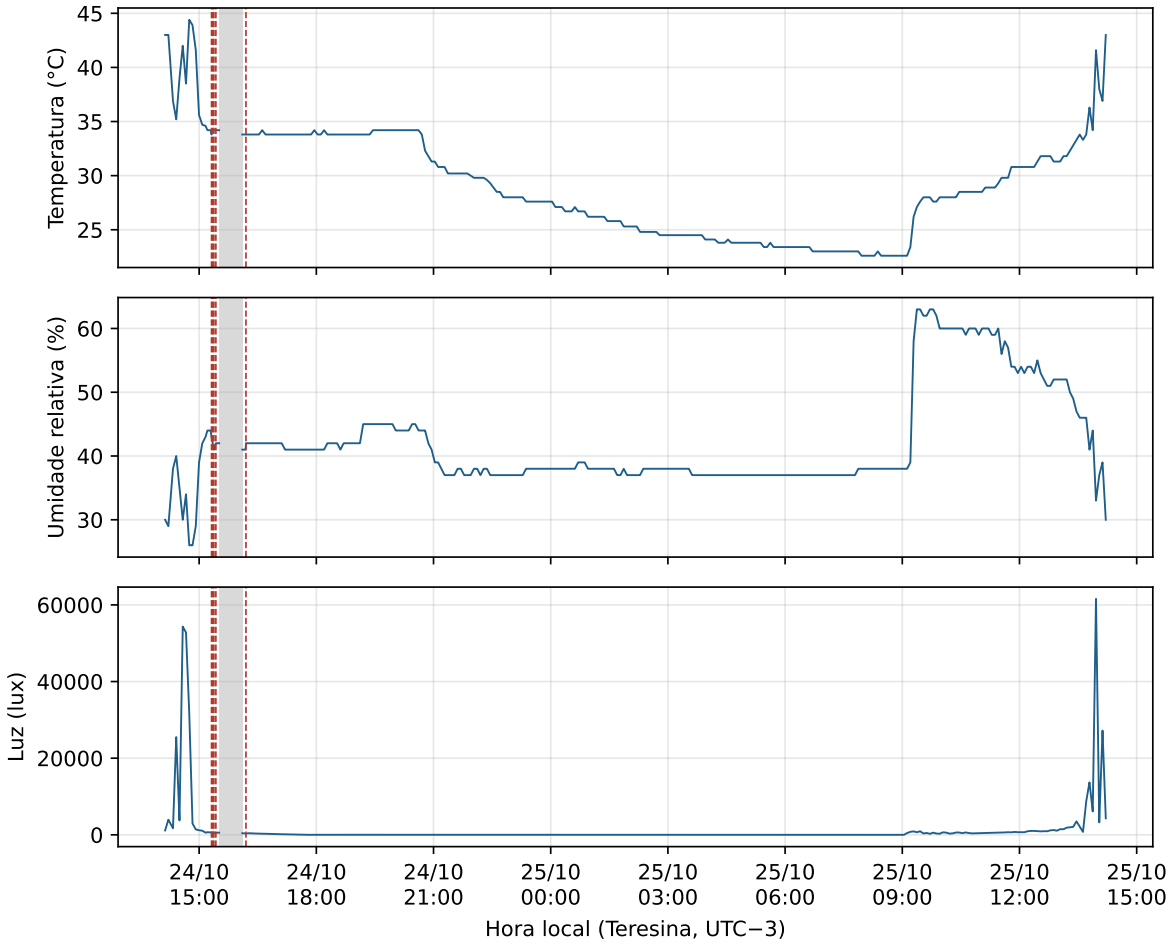


Figura 4 – Temperatura, umidade relativa e luz durante a campanha, com as lacunas à vista

Fonte: Dados coletados pela equipe de 24/10/2026, 14:07, a 25/10/2026, 14:13.

O que a série mostra:

- **Ciclo diário:** a luz e a temperatura têm pico perto das 14 h nos dois dias, quando o sol entra direto pela janela; a máxima foi de 44,4 °C às 14:44 de 24/10 e a mínima, de 22,6 °C, às 07:57 de 25/10, com o ar-condicionado ligado desde as 20:34. A amplitude foi de 21,8 °C e de 37 pontos de UR (26 a 63 %). Parte dela não é do ambiente: ao sol direto, o DHT11 aquece, e as máximas acima de 40 °C são do sensor aquecido.
- **Luz:** máximo de 61.571 lux às 13:57 de 25/10; acima de 20.000 lux só em 6 leituras, entre 14:24 e 14:44 de 24/10 e entre 13:57 e 14:07 de 25/10. Nenhum dia de calendário ficou completo; as 24,1 h somaram cerca de 2,0 mol/m² (1,06 em 24/10, das 14:07 à meia-noite, e 0,93 em 25/10, até as 14:13), luz de um ponto de meia-sombra dentro de casa.
- **Lacunas e reinícios:** 1 lacuna, de 35 min 39 s, a do **CT-16** (Tabela 16); 5 reinícios, todos provocados no **CT-17** e no **CT-18**, concentrados entre 15:19 e 16:12 de 24/10.
- **Eventos anotados:** o fim do sol direto às 14:55 de 24/10 (a luz cai de 31.712 lux para menos de 1.500 lux); a lâmpada do quarto, acesa às 17:53, que segura a luz em 15 lux até ser apagada às 00:28; o ar-condicionado ligado às 20:34, que leva a temperatura de 34,2 °C para 30,2 °C em uma hora e a UR de 45 para 37 %; e a janela aberta às 09:05 de 25/10, que leva a UR de 39 para 60 % em uma hora e a temperatura de volta para 28 °C.

Resultados que já têm número e procedência, independentes da campanha:

- pacote de 19 B, 5.472 B por dia, dentro dos 20 B de uma notificação com o MTU padrão (**CT-23**);
- *firmware* com 36.108 B de RAM (11 %) e 616.185 B de *flash* (47 %), compilado pelo PlatformIO;
- 10 de 10 testes automáticos do gravador e do `qualidade.py` passando, incluindo os três pacotes de exemplo do README e um arquivo sintético com perdas, repetida, reinício e leitura extra conhecidos;
- luz acumulada preservada através de uma desconexão de cerca de 55 min (**CT-20**, nRF Connect);
- teto do BH1750 de cerca de 121.557 lux com MTreg = 31, contra 54.612 lux na sensibilidade de fábrica (**DP-2**, FOLHA DE DADOS).

7.2 Requisitos atendidos

Placar: 15 de 17 requisitos atendidos, 1 atendido em parte, 1 não verificado e nenhum não atendido (Tabela 5). Antes da campanha, 9 estavam atendidos pelos ensaios com o nRF Connect, pelos ensaios cronometrados com o `ensaio.py`, pelo ensaio de alcance e pela inspeção (**RF-02**, **RF-06**, **RF-07**, **RF-08**, **RF-09**, **RF-12**, **RNF-04**, **RNF-06** e **RNF-09**), 1 não verificado (**RNF-01**: sem medidor USB, a corrente é só estimativa) e 7 dependiam da campanha ou dos ensaios com o gravador. Desses 7, 6 foram atendidos (**RF-01**, **RF-05**, **RF-11**, **RNF-05**, **RNF-07** e **RNF-08**) e 1 em parte (**RF-10**).

O atendido em parte é o **RF-10**: o gravador grava antes de confirmar e reconectou sozinho em todas as quedas (**CT-15**), mas, quando ele para, o CSV fica com buraco (**CT-16**), porque o nó não guarda temperatura e umidade (**DP-8**); só a luz atravessa. O não verificado é o **RNF-01**: a equipe não tem medidor USB, e a corrente segue estimada. Dois pontos ficaram como ressalva em requisitos atendidos: no **RF-11**, a primeira medida depois de ligar se perde quando o gravador demora a reconectar; no **RNF-08**, o vigia de

630 s para o nó travado com o enlace de pé não foi provocado na campanha.

7.3 Discussão

1. **O sistema responde à pergunta de projeto?** Em parte, e mais do que na AP1. O nó agora mede a luz em lux e entrega as três grandezas a quem guarda o dado, com hora e com as perdas contadas. A classificação do ambiente depende do modelo da AP3; a comparação planta a planta, com os valores recomendados de cada uma num banco de dados, e o aviso dependem do aplicativo, que ainda não fala com o nó. A campanha mostrou um ambiente em que o sol direto dura cerca de meia hora por dia, perto das 14 h, e em que a luz do resto do dia fica abaixo de 3.500 lux; a temperatura variou mais pelo que a equipe fez no quarto (janela, ar-condicionado) do que pelo dia lá fora. A campanha mediu o ambiente do jardim, sem uma planta específica, que é também o alvo da AP3.
2. **A tecnologia escolhida se sustentou?** Sim. A 5 m do notebook, o RSSI ficou entre −71 e −63 dBm, o enlace não caiu nenhuma vez sem ser provocado em quase 22 h seguidas, e a completude foi de 97,6 %, com as 7 janelas vazias todas na lacuna provocada. Deixaria de valer com o central longe do nó, além dos 20 a 25 m medidos no **CT-21**, ou desligado. O ponto fraco já conhecido é que BLE exige um central perto e ligado: tudo o que acontece longe dele se perde, exceto a luz.
3. **O conjunto de dados responde ao que a equipe quer fazer na AP3?** Ainda não. São 24,1 h e nenhum dia de calendário completo, com 6 eventos anotados; cada condição que o classificador precisa separar (sol forte, calor, ar seco, noite) aparece uma vez só, e frio e ar úmido não aparecem. O conjunto valida o caminho do dado e os atributos; o treino pede vários dias seguidos (Seção 5.8).
4. **Decisão mais acertada e decisão que a equipe refaria.** A mais acertada foi **DP-8** junto com **DP-6**: a luz acumulada e a confirmação da aplicação tornam a soma de luz imune a perdas e repetições, e a sequência transforma cada lacuna em "quantas medidas faltaram". A equipe refaria a ausência de fila local na **DP-8**: 5,5 kB de RAM teriam evitado o buraco de temperatura e umidade no **CT-16**.
5. **Se a autonomia real for metade da calculada, o projeto ainda faz sentido?** Na tomada, a pergunta não se aplica. Numa bateria 18650 de 2.600 mA h, a conta do Apêndice B dá menos de dois dias; pela metade, menos de um. O produto com bateria só faz sentido com o nó dormindo entre as medidas e acordando para anunciar, o que muda o contrato: o central passa a ter uma janela curta para conectar.

8 Limitações e próximo ciclo

Rubrica da AP2: comunicação dos resultados — 20 %

8.1 Limitações conhecidas

- **Sem central, sem temperatura e umidade.** O nó não tem fila (**DP-8**). Toda lacuna do conjunto de dados em que o gravador estava parado ou longe é uma lacuna real de temperatura e umidade; só a luz atravessa.
- **A hora é do central.** Nenhuma leitura tem hora do nó (**DP-10**). O erro é pequeno com notificação, mas o conjunto de dados depende do relógio do computador do gravador estar certo.
- **Sem segurança.** Sem pareamento, qualquer central ao alcance lê as leituras e pode escrever comandos, inclusive confirmar um pacote que não gravou (Seção 5.4).
- **Um central por vez.** Com o nRF Connect conectado, o gravador não conecta. No produto, isso impede dois celulares da mesma casa de acompanharem o mesmo jardim.
- **Campanha com o gravador, não com o aplicativo.** O contrato é o mesmo, mas o aplicativo FitoSfera ainda não foi exercitado com o nó.
- **Precisão dos sensores.** DHT11 com ±2 °C e passos de 1 % UR; BH1750 com ±20 % e resolução menor em luz fraca por causa do MTreg = 31.
- **Luz amostrada a cada 5 min.** A integral é grosseira em dias de nuvens passando.
- **Número mais frágil do relatório:** a corrente média do nó com o BLE ligado (cerca de 46 mA), que continua sendo estimativa: a equipe não tem medidor USB. Ela responde por cerca de 97 % do orçamento de energia.
- **O que a campanha revelou.** A coleta foi real, num quarto, e durou 24,1 h: um único ciclo diário, sem nenhum dia de calendário completo. O ambiente foi mexido pela equipe (ar-condicionado, janela, cortinas, lâmpada), e isso aparece na série mais do que o dia lá fora. O DHT11 aquece ao sol direto, e as leituras acima de 40 °C são do sensor, não do ar. A primeira medida depois de ligar o nó se perde se o gravador demora a reconectar. E dois problemas do lado do computador só apareceram na prática: um processo em segundo plano ignora o Ctrl+C, e o BlueZ pode segurar a conexão de um gravador morto, o que impede o nó de anunciar.

8.2 O que entra no próximo ciclo

Na AP3 a equipe vai treinar e embarcar um classificador da condição do ambiente do jardim como um todo, e não de uma planta específica: a cada leitura, adequado ou em qual condição de estresse (calor, frio, ar seco, ar úmido, pouca luz, sol forte). Sobre essa classificação, o aplicativo compara o ambiente planta a planta com os valores recomendados de cada uma, guardados num banco de dados. Com o modelo embarcado, parte dessa comparação pode descer para o próprio nó: por exemplo, o aplicativo envia ao nó as faixas das plantas cadastradas e o nó avisa sozinho, mesmo sem central por perto, quando o ambiente sai delas. Isso revê a **DP-3**, que hoje deixa toda a decisão no aplicativo. Os rótulos vêm de faixas gerais de cultivo da literatura e do registro de eventos; a hora do dia

e a luz integrada do dia (DLI) entram como atributos. A luz do dia exige dias completos, e o conjunto formado neste ciclo tem 24,1 h com 97,6 % de completude, mas nenhum dia de calendário com mais de 90 % das janelas; não é suficiente para treinar, porque cada condição aparece uma vez só, e a AP3 começa por uma campanha de vários dias. Antes do modelo, três mudanças no nó e no contrato:

1. uma fila circular na RAM (um dia, cerca de 5,5 kB), entregue em lote quando o central assina;
2. pareamento com *bonding*, com escrita no comando só para o central pareado;
3. o adaptador BLE no aplicativo FitoSfera, seguindo o contrato versão 2, para que a campanha da AP3 rode no celular e não num computador.

8.3 Divisão do trabalho

Os papéis do Módulo 2 sugeridos pela disciplina são integração, *firmware* e comunicação, dados (gravador, campanha e qualidade) e documentação. Na AP1, Lucas foi hardware, Gabriel foi *firmware* e Matheus foi integração e documentação; ninguém repete o papel na AP2.

- **Lucas Vilarinho** (*firmware* e comunicação): produziu o código do *firmware* e estudou as opções de comunicação, escolhendo o BLE por se alinhar ao aplicativo planejado. O código do BLE entrou no repositório pelo *commit* `122c41e`, enviado da conta do Gabriel.
- **Gabriel Araújo** (documentação): estudou o que foi produzido, reuniu no SIGAA os materiais da disciplina e, com os dados gerados pelo grupo, produziu o relatório.
- **Matheus Carneiro** (dados e integração): organizou os dados da campanha e os integrou ao trabalho.

Integrante	**Matrícula**	**Papel no ciclo**
Lucas Vilarinho	20249011769	*firmware* e comunicação
Gabriel Araújo	20249022010	documentação
Matheus Carneiro	20249012336	dados e integração

Referências

Adafruit Industries. *DHT sensor library for Arduino*. 2023. Repositório GitHub. Versão 1.4.6. Disponível em: <https://github.com/adafruit/DHT-sensor-library>. Citado 2 vezes nas paginas 23 e 50.

AL-FUQAHA, A. et al. Internet of Things: a survey on enabling technologies, protocols, and applications. *IEEE Communications Surveys & Tutorials*, v. 17, n. 4, p. 2347–2376, 2015. Citado 2 vezes nas paginas 11 e 16.

AOSONG ELECTRONICS. *DHT11: temperature and humidity module — product manual*. Guangzhou. Citado 2 vezes nas paginas 17 e 51.

BLIDH, H. *Bleak: Bluetooth Low Energy platform agnostic client*. 2025. Biblioteca Python. Disponível em: <https://github.com/hbldh/bleak>. Citado na pagina 25.

Espressif Systems. *ESP32 series datasheet*. [S.l.], 2023. Versão 4.3. Disponível em: <https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf>. Acesso em: 10 set. 2026. Citado 8 vezes nas paginas 18, 23, 28, 29, 33, 50, 51 e 55.

h2zero. *NimBLE-Arduino*. 2024. Repositório GitHub. Versão 1.4. Disponível em: <https://github.com/h2zero/NimBLE-Arduino>. Citado na pagina 23.

ROHM SEMICONDUCTOR. *BH1750FVI: digital 16bit serial output type ambient light sensor IC*. Kyoto, 2011. Citado 3 vezes nas paginas 17, 50 e 51.

Apêndices

APÊNDICE A – Mapa de barramento

O mapa mostra os dispositivos que estão no protótipo da AP2, conforme `firmware/src/main.cpp`. Em relação à AP1, saíram o módulo fotorresistor (GPIO19) e os três LEDs (GPIO21, 22 e 23), e entrou o BH1750 no I²C.

Tabela 19 – Mapa de barramento do protótipo

Dispositivo	**O que mede ou faz**	**Barramento**	**Como é selecionado**	**Pinos na DevKit V1**	**Tensão**	**Leituras por segundo**
Módulo DHT11	temperatura e UR do ar	serial de um fio, próprio do DHT (não é o 1-Wire da Dallas)	ponto a ponto: o pino é a seleção	D18 (GPIO18), DATA bidirecional	3,3 V	1/300 (até 4 a mais por ciclo, nas tentativas)
BH1750 (GY-302)	luz, em lux	I²C, 100 kHz	endereço `0x23` (ADDR solto)	SDA/SCL (21/22)	3,3 V	1/300
Rádio BLE (interno)	enlace com o central	interno ao ESP32	—	antena da placa	—	1 notificação a cada 300 s
Monitor serial	linha CSV de depuração	UART0, 115.200 Bd 8N1	ponto a ponto, pelo conversor USB-serial	TX0/RX0 (GPIO1/3)	3,3 V	1/300

Fonte: Elaborada pela equipe.

Decisões de barramento

1. Conflitos de pino encontrados, e como foram resolvidos.
O conflito previsto na AP1, o I²C padrão (21/22) ocupado pelos LEDs vermelho e amarelo, desapareceu com a saída dos LEDs (**DP-3** e **DP-5**). O barramento usa os pinos padrão com `Wire.begin(21, 22)`. Os três GPIO usados não definem o modo de inicialização, não estão ligados à *flash* e não são a UART0 (Espressif Systems, 2023) (**CT-10**).
2. Colisões de endereço, se há dois dispositivos I²C iguais.
Nenhuma: o BH1750 é o único dispositivo no barramento. Um segundo BH1750 iria para `0x5C`, com o pino ADDR em nível alto; um BME280, a troca prevista para o DHT11, fica em `0x76` ou `0x77`. O *firmware* varre os endereços de 1 a 126 ao ligar e os lista na serial, o que torna uma colisão visível na primeira linha.
3. O dispositivo mais lento do conjunto, e quanto ele custa ao laço.
Em tempo de espera, o BH1750: cada medida espera 100 ms com `delay()` pela conversão, que no modo de alta resolução leva tipicamente 120 ms com MTreg = 69 e cai para cerca de $120 \times 31/69 \approx 54$ ms com MTreg = 31 (ROHM SEMICONDUCTOR, 2011). Em barramento, o DHT11: cerca de 25 ms por leitura, dos quais cerca de 4,1 ms com as interrupções desligadas (a conta da AP1, a partir do código da biblioteca (Adafruit Industries, 2023)). Somados, são cerca de 125 ms a cada 300 s, ou 0,04 % do tempo. O laço não espera pela leitura seguinte (Listagem 4.1), e os *callbacks* do NimBLE rodam na tarefa do BLE, então nenhuma dessas esperas derruba o enlace.

APÊNDICE B – Orçamento de energia

O orçamento descreve o nó da AP2: sempre ligado, num carregador USB, com a CPU a 80 MHz e o BLE anunciando ou conectado o tempo todo, num ciclo de 300 s (288 ciclos por dia). Os estados são os da Seção 4.2. A inicialização acontece uma vez por energização e não entra na conta; a nova tentativa tem o mesmo consumo de *mede*. Os LEDs de aviso da AP1 saíram. O nó não dorme.

Tabela 20 – Orçamento de energia do nó (alimentação USB)

Estado	O que está ligado	Corrente (mA)	Duração por ciclo (s)	Ciclos /dia	Carga (mA·h /dia)	Origem do número
aguarda	ESP32 a 80 MHz com BLE (45), DHT11 em repouso (0,1), BH1750 desligado (<0,01), LED do módulo DHT11 (1,3)	46,4	299,873	288	1.113,2	ESTIMATIVA (README; FOLHA DE DADOS)
mede	como *aguarda*, com o DHT11 medindo (1,5) e o BH1750 convertendo (0,2)	48,0	0,125	288	0,5	ESTIMATIVA
envia	pico de transmissão BLE a 0 dBm (130), por cerca de 2 ms	131,4	0,002	288	0,02	FOLHA DE DADOS (pico); duração ESTIMATIVA
dorme	não existe: o nó não dorme	—	—	0	0,0	—
			Total diário orçado		**1.113,7**	
			Corrente média medida (**CT-07**)		—	não medida

Fonte: Elaborada pela equipe. Corrente do rádio de Espressif Systems (2023).

Origem de cada parcela:

- **ESP32 com BLE, 45 mA:** topo da faixa de 35 a 45 mA estimada no README para a CPU a 80 MHz com o BLE ativo. É a linha que mudou de natureza na AP2: na AP1 era a CPU com o rádio desligado; agora é a CPU com o rádio ligado o tempo todo. Não foi medida: a equipe não tem medidor USB.
- **Rádio:** a transmissão em si quase não pesa. Um pacote de 19 B ocupa o ar por frações de milissegundo; mesmo supondo 2 ms a 130 mA (Espressif Systems, 2023), são 0,02 mA h por dia. O que pesa é manter o rádio pronto para anunciar e para os eventos de conexão, e isso já está nos 45 mA de *aguarda*. A linha do rádio deixou de ser a estimativa de WiFi da AP1 (pico de 310 mA): os tempos do enlace estão medidos na Tabela 13, e a corrente é a da folha de dados.
- **DHT11:** 1,5 mA medindo e 0,1 mA em repouso, ordem de grandeza da folha de dados (AOSONG ELECTRONICS,); LED do módulo, 1,3 mA, a mesma suposição da AP1.
- **BH1750:** cerca de 0,12 mA convertendo e 1 µA desligado (ROHM SEMICONDUCTOR, 2011); no modo de uma medida o chip se desliga sozinho depois de cada conversão.

Quatro linhas de decisão

1. Consumo diário total e consumo médio.
Orçado: 1.113,7 mA h por dia, ou 1.113,7/24 ≈ 46,4 mA de média. Na AP1 eram 1.577,2 mA h (65,7 mA): a saída dos três LEDs de aviso e do LED do fotorresistor tirou mais do que o BLE acrescentou, no papel. A comparação com uma medição fica para a AP3, porque a equipe não tem medidor USB (**CT-07**).
2. Fonte de alimentação escolhida, e por quê.
Carregador USB comum de 5 V e 1 A ou mais, na tomada. O nó mede o ambiente de plantas dentro de casa, perto de uma tomada. A média de 46 mA é cerca de 9 % de uma porta USB 2.0 e o pico de 130 mA, cerca de 26 %. Um carregador rápido foi descartado porque religava a saída com carga baixa (Capítulo 6, "O que falhou"); pelo mesmo motivo, muitos *power banks* desligam sozinhos abaixo de 50 a 100 mA.
3. Autonomia estimada, e a conta que levou até ela.
Na tomada, ilimitada. Como referência, uma 18650 de 2.600 mA h com 80 % de profundidade de descarga dá 2.080 mA h úteis, e 2.080 ÷ 1.113,7 ≈ 1,9 dia. O regulador da placa é linear, então a corrente da bateria é praticamente a da carga.
4. O que acontece quando a energia acaba — ou enquanto o nó dorme.
O nó apaga por completo e perde a luz acumulada. Não há atuador nem estado a proteger. Diferente da AP1, a falta de energia agora é percebida por quem está longe do nó: o gravador anota `desconectado` em poucos segundos (**CT-18**), e, quando a energia volta, o id do reinício novo mostra no conjunto de dados que houve queda. O nó não dorme.

Se a conta atravessar tensões diferentes

Não é o caso: o ESP32 e os dois módulos estão no barramento de 3,3 V, alimentado pelo regulador linear da placa a partir dos 5 V da USB. Num regulador linear a corrente de entrada é a de saída mais a corrente própria do regulador, então os mA h da tabela valem também do lado dos 5 V.

Se o projeto for alimentado por painel

Não se aplica: o nó fica dentro de casa, na tomada.

Quem está dimensionando este sistema

O ESP32 acordado com o rádio responde por cerca de 97 % do consumo orçado ($45 \times 24 =$ 1.080 mA h de 1.113,7 mA h); o LED do módulo DHT11 por cerca de 3 %; os dois sensores, juntos, por menos de 0,1 %. A transmissão dos pacotes não aparece na conta. A decisão que dimensiona o sistema é a **DP-11**, com o rádio ligado o tempo todo à espera do central; uma versão a bateria teria de dormir entre as medidas e anunciar só por alguns segundos.

APÊNDICE C – Plano de testes

Um quadro por caso de teste. Os ensaios com o gravador usam `gravador/gravador.py` gravando em `dados/`, e os números saem do comando abaixo. Os registros seriais usam o monitor do PlatformIO, rodado na pasta `firmware/`. As evidências ficam na pasta `ensaios/` do repositório. Para um ensaio isolado, o gravador pode gravar num arquivo próprio, para não misturar o ensaio com a campanha.

```
python gravador/qualidade.py dados/mensagens.csv --eventos
    dados/eventos.csv
pio device monitor -b 115200 -f time -f log2file
python gravador/gravador.py --saida ensaios/ct17.csv --eventos
    ensaios/ct17-eventos.csv
```

Tabela 21 – CT-01 — período de leitura

Requisito verificado	**RF-01**
Objetivo	Verificar que as leituras do ciclo saem a cada 300 s ± 10 %.
Material	`dados/mensagens.csv` da campanha.
Procedimento	1. Rodar o `qualidade.py` sobre o arquivo final. 2. Ler a linha `intervalo entre leituras` (mediana, mínimo e máximo, sem as leituras extras).
Resultado esperado	Mediana entre 270 s e 330 s.
Resultado obtido	Mediana 300,2 s; mínimo 32,5 s (uma reconexão logo depois de uma leitura do ciclo); máximo 2.138,6 s, a lacuna do **CT-16**.
Veredito	**atende**
Evidência	Saída do `qualidade.py` no Apêndice F.

Fonte: Elaborada pela equipe.

Tabela 22 – CT-02 — luz sob sol direto

Requisito verificado	**RF-02**
Objetivo	Verificar que o BH1750 não satura sob sol pleno com MTreg = 31.
Material	Nó, nRF Connect ou gravador, um dia de sol.
Procedimento	1. Levar o nó ao sol direto, perto do meio-dia. 2. Pedir uma leitura com `01` e anotar o lux. 3. Repetir 3 vezes.
Resultado esperado	Lux válido (diferente de `0xFFFFFFFF`) acima de 54.612 lux, e abaixo de 121.557 lux.
Resultado obtido	Nó junto à janela, ao sol direto, em 24/10 às 14h06: 70 619, 70 682 e 70.530 lux. No teste de fumaça, 2 min antes, 71.803 lux.
Veredito	**atende**
Evidência	`ensaios/ct13-14-02.csv`, linhas do CT-02.

Fonte: Elaborada pela equipe.

Tabela 23 – CT-05 — formato CSV na serial

Requisito verificado	**RF-05**
Objetivo	Verificar que as linhas de dados da serial são CSV válido com 7 campos.
Material	Nó no cabo USB, monitor serial, Python.
Procedimento	1. Registrar 10 linhas de dados (pedir leituras com `01` para não esperar 50 min). 2. Tirar as linhas que começam por `#`. 3. Ler como CSV, com a linha de cabeçalho do `setup()`.
Resultado esperado	10 de 10 linhas com 7 campos.
Resultado obtido	Em 25/10, 14h20, com o nó na USB do notebook e o `gravador/ct05.py` pedindo 10 leituras pelo BLE: cabeçalho presente e 11 de 11 linhas de dados com 7 campos, nenhuma linha `#` depois do cabeçalho. Antes dele só há a saída da ROM no boot. O script abre a serial com DTR e RTS soltos: ativos, eles seguram o ESP32 em reset.
Veredito	**atende**
Evidência	`ensaios/ct05.log`

Fonte: Elaborada pela equipe.

Tabela 24 – CT-06 — falha de sensor

Requisito verificado	**RF-06**
Objetivo	Verificar que o nó tenta de novo, envia com campos ausentes e volta sozinho.
Material	Nó, monitor serial, nRF Connect.
Procedimento	1. Com o nó medindo, soltar o fio amarelo (DATA do DHT11) da porta D18. 2. Pedir uma leitura com `01`. 3. Conferir na serial as linhas `# nova tentativa em 5 s` e o pacote enviado. 4. Reconectar o fio e pedir outra leitura.
Resultado esperado	3 tentativas a cada 5 s; pacote com `0x8000` e `0xFFFF` nos campos do DHT11 e lux válido; leitura normal depois de reconectar, sem reiniciar.
Resultado obtido	Conforme o esperado, no ensaio com o nRF Connect registrado no README (“nova tentativa em 5 s quando um sensor falha”).
Veredito	**atende**
Evidência	README, seção “Estado da validação”.

Fonte: Elaborada pela equipe.

Tabela 25 – CT-07 — corrente média na USB

Requisito verificado	**RNF-01**
Objetivo	Medir a corrente média do nó com o BLE ligado e comparar com o orçado.
Material	Medidor USB em série, nó, gravador conectado.
Procedimento	1. Ligar o nó pelo medidor USB. 2. Anotar a corrente média por 60 s sem central (anunciando). 3. Repetir com o gravador conectado.
Resultado esperado	Até 100 mA nos dois casos; perto de 46 mA orçados.
Resultado obtido	Não medido: a equipe não tem medidor USB. O valor do Apêndice B (46 mA) continua sendo estimativa.
Veredito	**não verificado**
Evidência	—

Fonte: Elaborada pela equipe.

Tabela 26 – CT-10 — pinos com restrição

Requisito verificado	**RNF-04**
Objetivo	Confirmar que nenhum GPIO usado tem restrição de inicialização, está na *flash* ou é a UART0.
Material	`firmware/src/main.cpp`, Apêndice A, folha de dados do ESP32 (Espressif Systems, 2023).
Procedimento	1. Listar os pinos definidos no código: `PINO_DHT` = 18, `PINO_SDA` = 21, `PINO_SCL` = 22. 2. Conferir cada um contra GPIO0, 2, 5, 12 e 15 (inicialização), GPIO6 a 11 (*flash*), GPIO1 e 3 (UART0) e GPIO34 a 39 (só entrada).
Resultado esperado	Nenhum pino usado nas listas.
Resultado obtido	Nenhum dos 3 pinos aparece nas listas; GPIO18, que é bidirecional, está fora da faixa só de entrada.
Veredito	**atende**
Evidência	Este quadro e a Tabela 19.

Fonte: Elaborada pela equipe.

Tabela 27 – CT-12 — pacote no contrato

Requisito verificado	**RF-07**
Objetivo	Verificar que os pacotes seguem o contrato de 19 B e são decodificados sem ambiguidade.
Material	`gravador/test_protocolo.py`, nRF Connect, `dados/mensagens.csv`.
Procedimento	1. Rodar `python3 -m unittest discover gravador`. 2. Ler a característica de leitura no nRF Connect e conferir o tamanho. 3. Na campanha, ler `fora do contrato` no `qualidade.py`.
Resultado esperado	Os 3 exemplos do README decodificados com os valores da tabela; 19 bytes no nRF Connect; 0 fora do contrato.
Resultado obtido	Testes automáticos: 5 de 5 do decodificador passando. nRF Connect: pacotes de 19 bytes. Gravador (teste de fumaça e ensaios): 19 de 19 pacotes com 19 B. Campanha: 0 fora do contrato em 288 pacotes.
Veredito	**atende**
Evidência	Saída do `unittest`; Apêndice F.

Fonte: Elaborada pela equipe.

Tabela 28 – CT-13 — medir agora e assinatura

Requisito verificado	**RF-08**
Objetivo	Verificar que o nó mede e envia na hora a pedido do central.
Material	nRF Connect ou gravador; cronômetro (ou os carimbos do gravador).
Procedimento	1. Conectar e assinar a notificação; anotar o instante e o do pacote que chega. 2. Escrever `01` no comando 10 vezes, com alguns segundos entre elas, anotando cada instante. 3. Conferir que cada pacote tem a sequência seguinte.
Resultado esperado	10 de 10 pacotes novos em até 2 s.
Resultado obtido	nRF Connect: a assinatura dispara um envio imediato, e cada `01` traz um pacote com a sequência seguinte (README). Com o `gravador/ensaio.py` (24/10, 14h05): a assinatura trouxe um pacote em 0,47 s; 10 de 10 `01` trouxeram pacote novo, sequências 8 a 17 sem salto, latência mediana 0,166 s e máxima 0,215 s.
Veredito	**atende**
Evidência	README, “Estado da validação”; `ensaios/ct13-14-02.csv`.

Fonte: Elaborada pela equipe.

Tabela 29 – CT-14 — confirmação

Requisito verificado	**RF-09**
Objetivo	Verificar que só a confirmação do pacote em espera zera o contador de segundos sem sucesso.
Material	nRF Connect.
Procedimento	1. Receber um pacote de sequência NN e não confirmar; pedir outro com `01` e anotar o contador. 2. Confirmar com `02 NN` o pacote recebido; pedir outro e anotar o contador.
Resultado esperado	Sem confirmação, o contador cresce; com ela, volta perto de zero no pacote seguinte.
Resultado obtido	Com o `gravador/ensaio.py` (24/10, 14h05): sem confirmar, o contador foi de 114 a 117 e 121 s; depois de `02` com a sequência errada, 124 s; depois de `02` com a sequência do pacote em espera, 3 s no pacote seguinte.
Veredito	**atende**
Evidência	README, "Estado da validação"; `ensaios/ct13-14-02.csv`; na campanha, a linha `seg. sem confirmacao` do `qualidade.py`.

Fonte: Elaborada pela equipe.

Tabela 30 – CT-15 — queda do enlace

Requisito verificado	**RF-10** (teste mínimo da AP2: "queda do WiFi")
Objetivo	Verificar que o gravador volta sozinho depois de uma queda do enlace, e medir o que se perdeu.
Material	Gravador rodando e conectado; terminal no computador do gravador.
Procedimento	1. Anotar a hora e rodar `bluetoothctl power off`. 2. Esperar 2 min e rodar `bluetoothctl power on`. 3. Não mexer no gravador. Anotar a hora do `conectado` seguinte no `eventos.csv`.
Resultado esperado	`desconectado`, buscas sem sucesso, `conectado` em até 90 s depois do `power on` e um pacote extra imediato; nenhuma leitura perdida se a queda for menor que 300 s desde a última leitura.
Resultado obtido	24/10: `power off` às 14:17:41; `desconectado` às 14:17:44,8; cinco tentativas com o adaptador desligado; `power on` às 14:19:41; `encontrado` às 14:19:47,9 e `conectado` às 14:19:48,9, 7,2 s depois, com o pacote extra 0,3 s depois. A sequência pulou de 22 para 24: o `power off` caiu 297 s depois da última leitura, e o ciclo seguinte aconteceu com o rádio desligado. Sem lacuna acima de 450 s.
Veredito	**atende**
Evidência	`dados/eventos.csv` e `ensaios/diario-automatico.txt`.

Fonte: Elaborada pela equipe.

Tabela 31 – CT-16 — o gravador para

Requisito verificado	**RF-10** (teste mínimo da AP2)
Objetivo	Ver o que se perde quando quem guarda o dado para.
Material	Gravador rodando.
Procedimento	1. Parar o gravador com Ctrl+C e anotar a hora. 2. Esperar pelo menos 15 min (três ciclos). 3. Rodar o gravador de novo, gravando no mesmo arquivo. 4. Rodar o `qualidade.py`.
Resultado esperado	Por projeto (**DP-8**): lacuna no CSV, a sequência mostrando cerca de 3 leituras que o nó fez sem ninguém ouvindo, e o primeiro pacote da volta trazendo a luz do intervalo inteiro. O critério da disciplina, "sem buraco no CSV", não é atendido.
Resultado obtido	Gravador parado às 15:31:15 e de volta às 16:06:18 (24/10). Lacuna de 35 min 39 s; 7 leituras perdidas pela sequência (4 a 10); primeiro pacote com `seg_sem_sucesso` = 2.138 s e 16.240 lux · min novos, média de 456 lux no intervalo, coerente com 569 e 419 lux antes e depois. A primeira tentativa (14:34) falhou porque o `kill -INT` foi ignorado pelo gravador em segundo plano; corrigido no `gravador.py`.
Veredito	**atende em parte**
Evidência	Saída do `qualidade.py`; `eventos.csv` com `fim_gravador` e `inicio_gravador`.

Fonte: Elaborada pela equipe.

Tabela 32 – CT-17 — o nó reinicia

Requisito verificado	**RF-11** (teste mínimo da AP2)
Objetivo	Verificar que um reinício fica identificável e não gera duplicata nem perda falsa.
Material	Nó, gravador.
Procedimento	1. Com o gravador conectado, apertar EN e anotar a hora. 2. Esperar o gravador reconectar e pelo menos 2 leituras. 3. Rodar o `qualidade.py`.
Resultado esperado	Id do reinício novo, primeira sequência 1 e acumulado 0; `reinicios do no` aumenta em 1; nenhuma repetida e nenhuma perda atribuída ao reinício.
Resultado obtido	Ensaio com o nRF Connect (README): id muda, sequência volta a 1, acumulado recomeça de 0. Com o gravador (24/10, 15:25:36): `desconectado` em 1,9 s, `conectado` em 5,6 s, id novo `6d2e`; o `qualidade.py` contou o reinício, 0 repetidas e 0 perdas falsas. A primeira sequência recebida foi a 2 (acumulado 31 lux·min): a seq 1 saiu antes da reconexão e conta como perda real.
Veredito	**atende**
Evidência	Saída do `qualidade.py`; teste automático `test_contagens`, com um reinício sintético.

Fonte: Elaborada pela equipe.

Tabela 33 – CT-18 — o nó morre sem avisar

Requisito verificado	**RNF-08** (teste mínimo da AP2)
Objetivo	Medir quanto tempo o gravador leva para perceber que o nó caiu.
Material	Nó, gravador, relógio do computador do gravador.
Procedimento	1. Com o gravador conectado, arrancar o cabo USB do nó e anotar a hora (`date +%T.%N` no terminal). 2. Ler a hora do `desconectado` no `eventos.csv`. 3. Repetir 3 vezes.
Resultado esperado	`desconectado` em até 10 s.
Resultado obtido	Quatro arranques em 24/10: 15:20:22,7 → 5,9 s; 15:22:16,5 → 3,6 s; em 15:18:41 e 16:11:31 o Enter saiu atrasado (num deles, depois do próprio `desconectado`), e as medidas não valem. Reconexão sozinha nas quatro.
Veredito	**atende**
Evidência	`ensaios/ct18-hora.txt` e `dados/eventos.csv`.

Fonte: Elaborada pela equipe.

Tabela 34 – CT-19 — antes do NTP: a hora das leituras

Requisito verificado	**DP-10** (teste mínimo da AP2)
Objetivo	Conferir de onde vem a hora das primeiras leituras depois de ligar o nó.
Material	Nó, gravador.
Procedimento	1. Ligar o nó com o gravador já rodando. 2. Conferir as primeiras 3 linhas no `mensagens.csv`.
Resultado esperado	Nenhuma leitura com hora do nó (o pacote não tem esse campo); todas com `ts_recebido` do gravador; a primeira chega pouco depois do `conectado`.
Resultado obtido	288 de 288 leituras sem hora do nó, todas com `ts_recebido` do gravador; depois de cada um dos 5 reinícios, o primeiro pacote chegou de 0,1 a 0,6 s depois do `conectado`.
Veredito	**atende**
Evidência	Primeiras linhas do `mensagens.csv`.

Fonte: Elaborada pela equipe.

Tabela 35 – CT-20 — luz através de uma desconexão

Requisito verificado	**RF-12**
Objetivo	Verificar que a luz de um período sem comunicação não se perde nem conta em dobro.
Material	nRF Connect; na campanha, o `mensagens_limpo.csv`.
Procedimento	1. Desconectar o central por pelo menos 30 min. 2. Reconectar e comparar a diferença dos acumulados com a luz e o tempo entre os pacotes.
Resultado esperado	A diferença cobre o período inteiro.
Resultado obtido	Depois de cerca de 55 min desconectado, o pacote seguinte trouxe a luz do período inteiro, e a diferença bateu com a luz e o tempo entre os pacotes (README).
Veredito	**atende**
Evidência	README, "Estado da validação"; coluna `luz_nova_lux_min` do arquivo limpo.

Fonte: Elaborada pela equipe.

Tabela 36 – CT-21 — alcance

Requisito verificado	**RNF-06**
Objetivo	Medir a distância máxima em que o enlace ainda funciona, dentro e fora da casa.
Material	Nó num carregador USB, a cerca de 1 m do chão; central conectado.
Procedimento	1. Com o central conectado e o nó no quarto fechado, afastar o central aos poucos, pedindo leituras. 2. Anotar a maior distância em que as leituras ainda chegam e quantas paredes há no caminho. 3. Repetir com o nó no jardim aberto, fora da casa.
Resultado esperado	Pelo menos 10 m dentro de casa, com paredes.
Resultado obtido	Cerca de 20 a 25 m dentro da casa, com várias paredes; cerca de 45 a 50 m no ambiente aberto, com uma parede.
Veredito	**atende**
Evidência	Ensaio da equipe; distâncias aproximadas, sem registro em arquivo. Na campanha, o RSSI de cada conexão fica no `qualidade.py` (Apêndice F).

Fonte: Elaborada pela equipe.

Tabela 37 – CT-22 — longa duração: a campanha

Requisito verificado	**RNF-05** e **RNF-07** (teste mínimo da AP2)
Objetivo	Verificar que o sistema roda sem ninguém olhando e forma um conjunto de dados completo.
Material	Nó instalado, gravador, `qualidade.py`.
Procedimento	1. Instalar o nó, iniciar o gravador e anotar no diário (Apêndice F). 2. Deixar correr pelo menos 24 h, anotando os eventos. 3. No fim, rodar o `qualidade.py` sobre o arquivo final, com `-inicio` e `-fim` declarados.
Resultado esperado	Pelo menos 24 h; completude de pelo menos 90 %; campos ausentes até 5 %; toda lacuna explicada.
Resultado obtido	24,1 h (24/10 14:07 a 25/10 14:13); completude de 97,6 % das janelas; 0 campos ausentes; 1 lacuna, de 35 min 39 s, provocada pelo **CT-16**.
Veredito	**atende**
Evidência	Seções 5.6 e 5.7; Apêndice F.

Fonte: Elaborada pela equipe.

Tabela 38 – CT-23 — tamanho do pacote

Requisito verificado	**RNF-09**
Objetivo	Confirmar que o pacote cabe numa notificação com o MTU padrão.
Material	`firmware/src/main.cpp`, nRF Connect.
Procedimento	1. Conferir `uint8_t pacote[19]` em `enviarPacote()`. 2. Ler a característica no nRF Connect sem negociar MTU.
Resultado esperado	Até 20 B.
Resultado obtido	19 bytes no código e no nRF Connect.
Veredito	**atende**
Evidência	Listagem do código; README.

Fonte: Elaborada pela equipe.

APÊNDICE D – Declaração de uso de inteligência artificial generativa

Tabela 39 – Uso de IA generativa neste ciclo

Etapa	**Ferramenta**	**Para quê**	**O que a equipe verificou**
Firmware BLE (*commit* `122c41e`)	Claude (Anthropic)	apoio na escrita do código do *firmware*	ensaios com o nRF Connect registrados no README
Revisão do repositório	Claude (Anthropic), no Claude Code	organizou o repositório para a AP2, movendo o projeto PlatformIO para `firmware/`	o *firmware* em `firmware/` foi compilado, gravado na placa e está funcionando
Gravador e análise de dados	Claude (Anthropic), no Claude Code	escreveu `gravador.py`, `protocolo.py`, `qualidade.py`, `limpar.py`, `grafico.py`, `ensaio.py`, `ct05.py` e os testes automáticos, a partir do contrato BLE do README; corrigiu o gravador durante a campanha (SIGINT e SIGTERM)	10 testes automáticos passando; a equipe rodou o gravador com o nó (teste de fumaça, ensaios e campanha de 24/10)
Ensaios e campanha	Claude (Anthropic), no Claude Code, no notebook do gravador	rodou o teste de fumaça, os ensaios cronometrados (**CT-02**, **CT-13**, **CT-14**), o **CT-05** e os ensaios automáticos **CT-15** e **CT-16**; iniciou, acompanhou e encerrou o gravador; registrou o diário a partir do que a equipe contou	as ações físicas foram da equipe: instalação e reposicionamento do nó, botão EN (**CT-17**), cabo USB (**CT-18**) e os eventos do quarto; os dados vêm do nó, sem edição
Relatório	Claude (Anthropic), no Claude Code	primeira versão do texto deste relatório, a partir do relatório da AP1, do README, do código e das instruções de entrega; depois da campanha, preencheu os números a partir das saídas do `qualidade.py` e dos arquivos de ensaio	a equipe revisou cada capítulo e forneceu os dados que só ela tinha (local, papéis, eventos, objetivo da AP3)
Não houve uso em	montagem do circuito e coleta dos dados pelo nó.		

Fonte: Elaborada pela equipe.

Declaração. A equipe declara que o conteúdo deste relatório e do código a ele associado foi revisado e é compreendido por todos os integrantes, e que todo uso de ferramentas de inteligência artificial generativa está registrado na tabela acima. Todos os integrantes leram esta declaração e confirmam a revisão do relatório.

Integrante	**Matrícula**	**Papel no ciclo**
Lucas Vilarinho	20249011769	*firmware* e comunicação
Gabriel Araújo	20249022010	documentação
Matheus Carneiro	20249012336	dados e integração

Teresina, 25 de outubro de 2026.

APÊNDICE E – Checklist de prontidão para a campanha

A equipe não localizou o checklist de prontidão de 06/10. A tabela abaixo foi montada a partir dos doze itens do modelo da disciplina, escritos para WiFi e MQTT: cada item traz o equivalente no BLE e a evidência no sistema de agora, e a coluna "Feito?" diz o estado antes da campanha. A última coluna é o que a campanha mostrou.

Tabela 40 – Checklist de prontidão e o que a campanha mostrou

#	Item (equivalente no BLE)	Evidência	Feito?	Na campanha
1	Laço sem `delay()` longo	Listagem 4.1; só 100 ms do BH1750 por ciclo	sim	nenhum travamento em 24,1 h; `01` respondido em 0,17 s
2	Keepalive: *timeout* de supervisão e vigia de silêncio	`SILENCIO_MAX_S` = 630 em `gravador.py`	sim	`desconectado` em 3,6 e 5,9 s (**CT-18**); o vigia de 630 s não precisou disparar
3	`online`/`offline`: `conectado`/`desconectado` no `eventos.csv`	`gravador.py`, `ao_desconectar`	sim	9 `conectado` e 6 `desconectado`, todos de ensaios
4	O gravador reconecta sozinho ao nó	laço `rodar()`, espera de 2 a 60 s	sim	sozinho em todas as quedas (mediana 20,1 s); uma vez precisou de `bluetoothctl disconnect`
5	Contrato com `id`, `boot`, `seq`, `ts`: endereço, id do reinício, sequência e hora de chegada	Tabela 11	sim	288 de 288 pacotes no contrato
6	Um reinício não gera duplicatas nem perdas falsas	`qualidade.py`; teste automático com reinício sintético	sim	5 reinícios, 0 repetidas, 0 perdas falsas; a seq 1 se perdeu em 4
7	A hora sai certa; o que acontece sem NTP	**DP-10**: hora do gravador em todas as leituras	sim	288 de 288 com hora do gravador
8	O que o nó faz sem central: fila e o que se perde	**DP-8**: sem fila; a luz atravessa, T e UR se perdem	não	como previsto: 7 leituras perdidas no **CT-16**, luz do intervalo inteira
9	Onde roda o gravador e o que acontece se ele parar	Tabela 10; **CT-16**	sim	lacuna de 35 min 39 s; na 1ª tentativa o gravador ignorou o Ctrl+C
10	Política de *flush* e o que se aceita perder	`fsync` a cada linha, antes da confirmação	sim	nenhuma linha truncada nas paradas do gravador
11	`secrets.h` fora do repositório	não se aplica: o BLE não tem credencial	—	—
12	Teste de fumaça sem perdas, ou com as perdas explicadas	`ensaios/fumaca.csv` e `ensaios/fumaca-eventos.csv`: conectado em 1,2 s, primeiro pacote 0,5 s depois, 19 B, 0 fora do contrato	sim	o gravador funcionou de primeira com o nó

Fonte: Elaborada pela equipe, a partir dos itens do modelo da disciplina, adaptados ao BLE.

O que o checklist não pegou

Duas falhas do lado do computador, que nenhum item previa. A primeira: o gravador rodando em segundo plano num script ignorou o `kill -INT`, porque um processo de fundo de um script sem controle de tarefas nasce com o SIGINT ignorado; o **CT-16** automático não parou nada. A segunda: encerrado à força, o gravador deixou a conexão aberta no BlueZ, e o nó, ainda conectado, parou de anunciar. Os itens 4 e 9 perguntam se o gravador reconecta e o que acontece se ele para, mas não como ele é parado. As duas foram corrigidas na campanha (Capítulo 6, "O que falhou").

APÊNDICE F – Registro da campanha de coleta

Diário da campanha

Tabela 41 – Diário da campanha de coleta

Data e hora	**O que aconteceu**	**Como a equipe soube**	**O que a equipe fez**
24/10 14:07	nó instalado num quarto, junto a uma janela aberta, a 50 cm do chão; gravador iniciado no notebook, a cerca de 5 m	—	—
24/10 14:17	**CT-15**: Bluetooth do notebook desligado por 2 min	diário automático e `eventos.csv`	nada: o gravador reconectou sozinho às 14:19:49
24/10 14:34	**CT-16** automático falhou: o `kill -INT` foi ignorado e o gravador seguiu gravando	conferência às 15:10, nas leituras que continuaram chegando	corrigiu o `gravador.py` e trocou o processo às 15:11
24/10 15:11	na troca, o BlueZ manteve a conexão do gravador antigo; o nó parou de anunciar	`nao_encontrado` no terminal	`bluetoothctl disconnect`; reconectou às 15:12:44, sem perda
24/10 15:18 a 16:11	**CT-18**: USB do nó arrancado 4 vezes (15:18, 15:20, 15:22, 16:11)	hora anotada no terminal; `eventos.csv`	religou o USB; o gravador reconectou sozinho nas quatro
24/10 15:25	**CT-17**: botão EN	hora anotada no terminal	nada: reconexão em 5,6 s
24/10 15:31 a 16:06	**CT-16** refeito: gravador parado por 35 min	diário automático	gravador de volta às 16:06:18
25/10 00:28	nó reposicionado (mesma altura, junto à janela, a cerca de 5 m)	anotação da equipe	nada: a conexão não caiu
25/10 09:05	nó de volta à posição original	anotação da equipe	—
25/10 14:12	fim da coleta (gravador encerrado às 14:12:37)	—	`qualidade.py` sobre o arquivo final

Fonte: Elaborada pela equipe.

Registro de eventos para rotulagem

O que aconteceu em volta do nó enquanto ele media, com hora: o sol que entrou direto no sensor, a sombra, a janela aberta ou fechada, a chuva, alguém mexendo no nó. É deste registro que sai a rotulagem no Módulo 3.

Tabela 42 – Eventos observados durante a coleta

Início	Fim	Evento	Quem anotou, e como
24/10 14:07	24/10 14:55	sol direto no sensor, janela aberta; DHT11 aquecido pelo sol (até 44,4 °C)	equipe, conferido nos dados
24/10 14:55	24/10 17:53	sem sol direto; luz natural caindo até o anoitecer	equipe, conferido nos dados
24/10 17:53	25/10 00:28	escuro lá fora; lâmpada do quarto acesa (15 lux)	equipe
24/10 20:34	25/10 09:05	janela e cortinas fechadas; ar-condicionado ligado	equipe
25/10 00:28	25/10 09:05	luz apagada; nó reposicionado no quarto	equipe
25/10 09:05	25/10 14:13	cortinas e janela abertas, ar-condicionado desligado, nó na posição original; sol direto a partir de cerca de 13:55	equipe, conferido nos dados

Fonte: Elaborada pela equipe.

Saída do relatório de qualidade

A saída completa do `qualidade.py` sobre o arquivo final, sem edição, gerada pelo comando abaixo, rodado na raiz do repositório. O quadro lê `docs/figuras/qualidade.txt`; enquanto o arquivo não existe, mostra um aviso de pendência.

```
python gravador/qualidade.py dados/mensagens.csv --eventos
   dados/eventos.csv \
    --inicio '2026-10-24 14:07' --fim '2026-10-25 14:13' >
       dados/qualidade.txt
cp dados/qualidade.txt docs/figuras/qualidade.txt
```

```
# qualidade.py sobre dados/mensagens.csv (eventos: dados/eventos.csv)

## Campanha
inicio                      24/10 14:07:00 (hora de Teresina)
fim                         25/10 14:13:00
duracao                     24h06min00s (24.10 h)
periodo nominal             300 s
leituras esperadas          290  (duracao / periodo, +1)

## Contagens
linhas no arquivo           288
fora do contrato            0  (tamanho diferente de 19 bytes)
repetidas (descartadas)     0
leituras unicas             288
  extras (fora do ciclo)    1  (assinatura ao reconectar ou comando
     0x01)
  no ciclo de 300 s         287
faltam (a sequencia pulou)  12  (o no mediu, ninguem recebeu)
reinicios do no             5
  24/10 15:19:03  novo id 86f1, primeira seq recebida 2
  24/10 15:20:44  novo id a483, primeira seq recebida 2
```

```
  24/10 15:22:38  novo id 56be, primeira seq recebida 1
  24/10 15:25:42  novo id 6d2e, primeira seq recebida 2
  24/10 16:11:54  novo id aef1, primeira seq recebida 2
leituras sem hora do no     288 de 288  (o no nao tem relogio; hora =
   chegada no gravador)

## Completude
janelas de 300 s c/ leitura 283 de 290  -> 97.6 %
unicas no ciclo / esperadas 287 de 290  -> 99.0 %

## Tempo
intervalo entre leituras    mediana 300.2 s; min 32.5 s; max 2138.6 s
seg. sem confirmacao        mediana 300 s; max 2138 s  (no pacote; ~300
   s = confirmacao anterior chegou)
  acima de 360 s            2  (a confirmacao anterior nao chegou ao no)

## Sensores
temperatura (C)    ausentes    0; fora de 0.0..50.0:   0; observada
   22.6 a 44.4
umidade (%)        ausentes    0; fora de 20.0..90.0:   0; observada
   26.0 a 63.0
luz (lux)          ausentes    0; fora de 0..121557:   0; observada 0 a
   61571

## Lacunas (intervalo > 450 s): 1
24/10 15:30:42 -> 24/10 16:06:21      35min39s  ~6 leituras  o no mediu
   7 vez(es) sem entregar; gravador parado
tempo total em lacunas      35min39s

## Luz integrada por dia (diferenca do acumulado, dia da chegada)
24/10      959040 lux.min  ~  1.06 mol/m2/dia  (117 leituras de 288 num
   dia)
25/10      841416 lux.min  ~  0.93 mol/m2/dia  (171 leituras de 288 num
   dia)

## Enlace BLE (do arquivo de eventos)
encontrado -> conectado     mediana 1.1 s; max 1.5 s; n = 9
conectado -> 1o pacote      mediana 0.3 s; max 0.6 s; n = 9
desconectado -> conectado   mediana 20.1 s; max 124.0 s; n = 6
RSSI ao encontrar           mediana -68 dBm; pior -127 dBm; melhor -63
   dBm; n = 9

## Eventos do gravador
encontrado              9
conectado               9
inscrito                9
desconectado            6
erro                    5
inicio_gravador         3
nao_encontrado          3
fim_gravador            2
```

Listing F.1 – Saída do `qualidade.py` sobre o arquivo final da campanha